```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline

#as the enconding was not default tahts why we have used latin-1
data=pd.read_csv("C:/Users/aj/Downloads/Mobiles Dataset
(2025).csv",encoding="latin-1")
data
```

```
    Company Name            Model Name Mobile Weight   RAM     Front
Camera  \
0          Apple       iPhone 16 128GB          174g   6GB
12MP
1          Apple       iPhone 16 256GB          174g   6GB
12MP
2          Apple       iPhone 16 512GB          174g   6GB
12MP
3          Apple  iPhone 16 Plus 128GB          203g   6GB
12MP
4          Apple  iPhone 16 Plus 256GB          203g   6GB
12MP
..           ...                    ...           ...   ...
...
925         Poco          Pad 5G 128GB          571g   8GB
8MP
926         Poco          Pad 5G 256GB          571g   8GB
8MP
927      Samsung  Galaxy Z Fold6 256GB          239g  12GB  10MP, 4MP
(UDC)
928      Samsung  Galaxy Z Fold6 512GB          239g  12GB  10MP, 4MP
(UDC)
929      Samsung    Galaxy Z Fold6 1TB          239g  12GB  10MP, 4MP
(UDC)

    Back Camera            Processor Battery Capacity  Screen Size  \
0          48MP           A17 Bionic         3,600mAh   6.1 inches
1          48MP           A17 Bionic         3,600mAh   6.1 inches
2          48MP           A17 Bionic         3,600mAh   6.1 inches
3          48MP           A17 Bionic         4,200mAh   6.7 inches
4          48MP           A17 Bionic         4,200mAh   6.7 inches
..          ...                  ...              ...          ...
925         8MP  Snapdragon 7s Gen 2        10,000mAh  12.1 inches
926         8MP  Snapdragon 7s Gen 2        10,000mAh  12.1 inches
927        50MP   Snapdragon 8 Gen 3          4400mAh   7.6 inches
928        50MP   Snapdragon 8 Gen 3          4400mAh   7.6 inches
929        50MP   Snapdragon 8 Gen 3          4400mAh   7.6 inches
```

```
    Launched Price (Pakistan) Launched Price (India) Launched Price
(China)  \
0                  PKR 224,999             INR 79,999              CNY
5,799
1                  PKR 234,999             INR 84,999              CNY
6,099
2                  PKR 244,999             INR 89,999              CNY
6,499
3                  PKR 249,999             INR 89,999              CNY
6,199
4                  PKR 259,999             INR 94,999              CNY
6,499
..                         ...                    ...
...
925                 PKR 66,220             INR 23,999              CNY
2,099
926                 PKR 71,220             INR 25,999              CNY
2,299
927                PKR 604,999            INR 164,999
¥13,999
928                PKR 544,999            INR 176,999             CNY
15,999
929              Not available            INR 200,999            CNY
17,999

    Launched Price (USA) Launched Price (Dubai)  Launched Year
0                 USD 799              AED 2,799           2024
1                 USD 849              AED 2,999           2024
2                 USD 899              AED 3,199           2024
3                 USD 899              AED 3,199           2024
4                 USD 949              AED 3,399           2024
..                    ...                    ...            ...
925               USD 280              AED 1,029           2024
926               USD 300              AED 1,099           2024
927             USD 1,899              AED 7,199           2024
928              USD 1719              AED 7,699           2024
929             USD 2,259              AED 8,699           2024

[930 rows x 15 columns]
```

```
#shape
data.shape
```

```
(930, 15)
```

```
#info
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 930 entries, 0 to 929
```

```
Data columns (total 15 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   Company Name                930 non-null    object
 1   Model Name                  930 non-null    object
 2   Mobile Weight               930 non-null    object
 3   RAM                         930 non-null    object
 4   Front Camera                930 non-null    object
 5   Back Camera                 930 non-null    object
 6   Processor                   930 non-null    object
 7   Battery Capacity            930 non-null    object
 8   Screen Size                 930 non-null    object
 9   Launched Price (Pakistan)   930 non-null    object
 10  Launched Price (India)      930 non-null    object
 11  Launched Price (China)      930 non-null    object
 12  Launched Price (USA)        930 non-null    object
 13  Launched Price (Dubai)      930 non-null    object
 14  Launched Year               930 non-null    int64
dtypes: int64(1), object(14)
memory usage: 109.1+ KB
```

```python
#drop columns that are not useful
data=data.drop(columns=["Launched Price (Pakistan)","Launched Price (China)","Launched Price (USA)","Launched Price (Dubai)"])

data.sample(3)
```

```
    Company Name       Model Name Mobile Weight  RAM Front Camera  \
258         Vivo        V19 256GB          176g  8GB         32MP
72         Apple  iPhone XS 256GB          177g  4GB          7MP
712        Tecno    Megapad 128GB          420g  6GB         12MP

     Back Camera                Processor Battery Capacity  Screen
Size  \
258         48MP  Qualcomm Snapdragon 712          4500mAh  6.44
inches
72   12MP + 12MP               A12 Bionic         2,658mAh   5.8
inches
712         50MP       MediaTek Helio G80         7,500mAh  10.9
inches

    Launched Price (India)  Launched Year
258             INR 31,990           2020
72             INR 109,900           2018
712             INR 24,999           2024
```

```python
#rename columns if needed
data.rename(columns={"Launched Price (India)":"price"},inplace=True)

#doing ydata profiling to get idea on data
from ydata_profiling import ProfileReport
```

```
pr=ProfileReport(data)
pr.to_file("mobile.html")
```

```
<IPython.core.display.HTML object>
```

{"model_id":"d34246a05eea48d19623dfe0b5525a1a","version_major":2,"version_minor":0}

```
  0%|
| 0/11 [00:00<?, ?it/s]
```

{"model_id":"b25d7872ac05489e99c5482b419940f2","version_major":2,"version_minor":0}

{"model_id":"ea1bf8f3bd5c48339dd3d467fa49e2c1","version_major":2,"version_minor":0}

{"model_id":"62b0450f99e2482e9d6fa6db4dc90ef5","version_major":2,"version_minor":0}

```
df=data.copy()
#checking missing value in data
data.isnull().sum()
#so in this dataset we dont have any missing value
```

```
Company Name        0
Model Name          0
Mobile Weight       0
RAM                 0
Front Camera        0
Back Camera         0
Processor           0
Battery Capacity    0
Screen Size         0
price               0
Launched Year       0
dtype: int64
```

```
#modifying some of varibale value like mobile weight,battery
capacity,RAM,Front Camera,Back Camera,price
df["Mobile Weight"]=df["Mobile Weight"].astype("string")
df["Mobile Weight"]=df["Mobile
Weight"].str.replace("g","").str.strip().astype(float)
#we would keep it in float

#for battery capacity
#check how many unique value it hold
df["Battery Capacity"].unique()
#if inconsistent spacing are there in this column so we would strip it
#convert it into str to strip it
df["Battery Capacity"]=df["Battery
```

```
Capacity"].astype("string").str.strip()
#remove mah from the data
df["Battery Capacity"]=df["Battery Capacity"].str.replace("mAh","")

#ram
df["RAM"]=df["RAM"].astype("string")
df["RAM"]=df["RAM"].str.extract(r'^(\d+)')
df["RAM"]=df["RAM"].astype(int)

df["Front Camera"]=df["Front Camera"].astype("string")

df["Front Camera"]=df["Front Camera"].str.extract(r'(\d+)')

df["Front Camera"]=df["Front Camera"].astype(int)

#back camera
df["Back Camera"]=df["Back Camera"].astype("string")
df["Back Camera"]=df["Back Camera"].str.extract(r'(\d+)')
df["Back Camera"]=df["Back Camera"].astype(int)

#battery capacity
df["Battery Capacity"]=df["Battery Capacity"].str.replace(",",'')

#remove inches
df["Screen Size"]=df["Screen Size"].str.extract(r'^([\d.]+)')

df["Screen Size"]=df["Screen Size"].astype(float)

#price
df["price"]=df["price"].str.replace("INR","")
df["price"]=df["price"].str.replace(",","")

#extracting gb ram as there are outlier in the data so we extarct tb
ram separetly
df["f"]=df["Model Name"].str.extract(r'(\d+)GB')

df.isnull().sum()
```

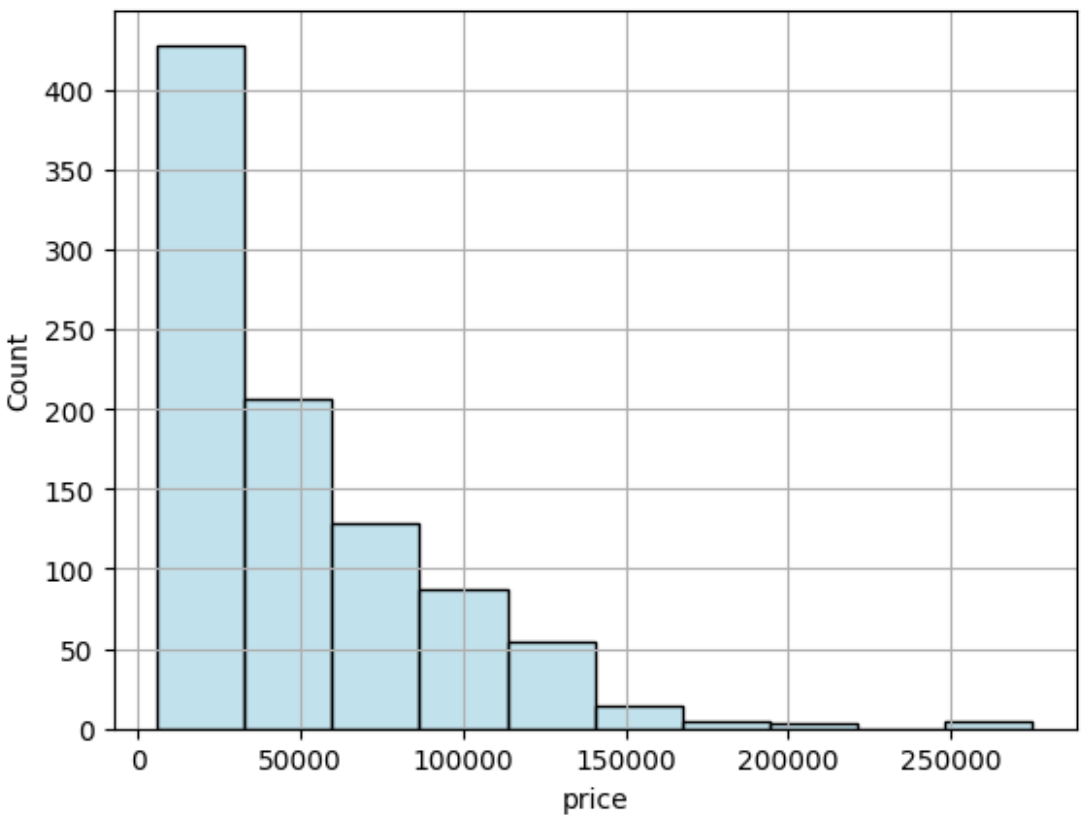
```
Company Name          0
Model Name            0
Mobile Weight         0
RAM                   0
Front Camera          0
Back Camera           0
Processor             0
Battery Capacity      0
Screen Size           0
price                 0
Launched Year         0
f                   157
dtype: int64
```

```
df["ftb"]=df["Model Name"].str.extract(r'(\d+)TB')

#convert tb into gb
df["ftb"]=df["ftb"].fillna(0)
df["ftb"]=df["ftb"].astype(int)
df["ftb"]=(df["ftb"]*1024)

df["storage"]=df["f"].combine_first(df["ftb"])
#now we can drop both "f" and "ftb"
df=df.drop(columns=["f","ftb"])

#drop model name
df=df.drop(columns=["Model Name"])

#replace in company name
df["Company Name"]=df["Company Name"].replace({"Poco":"poco"})

df["Company Name"]=df["Company Name"].replace({"POCO":"poco"})

df
```

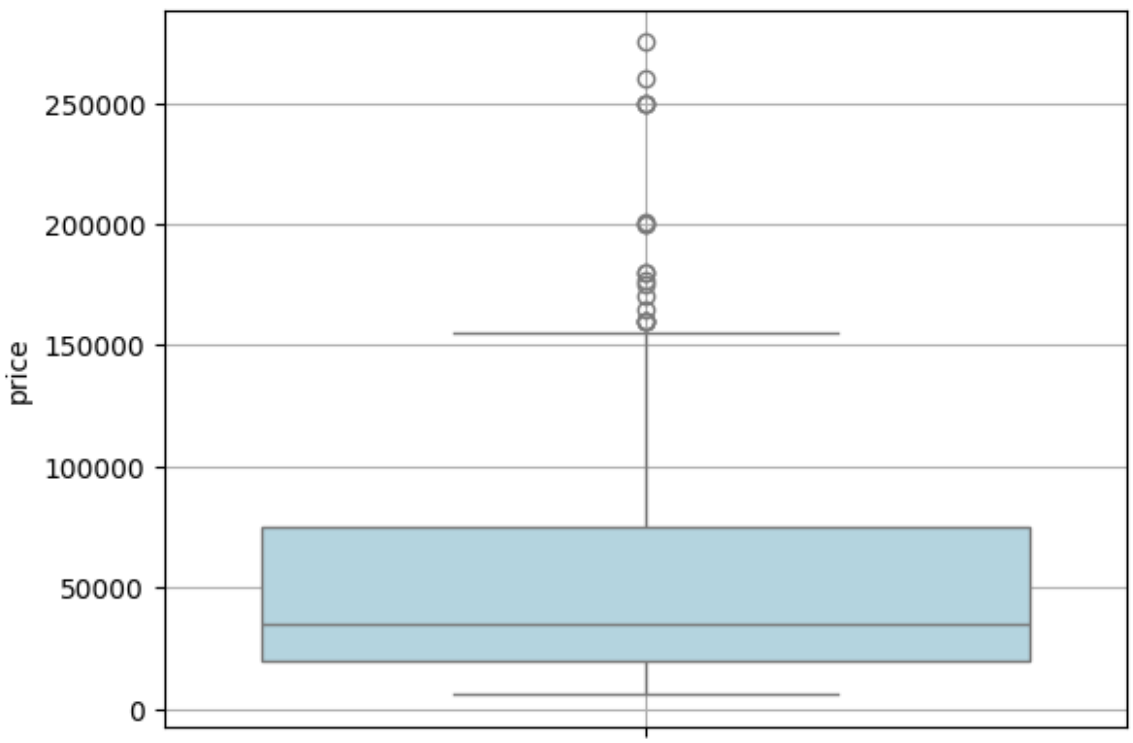
```
    Company Name  Mobile Weight  RAM  Front Camera  Back Camera  \
0          Apple          174.0    6            12           48
1          Apple          174.0    6            12           48
2          Apple          174.0    6            12           48
3          Apple          203.0    6            12           48
4          Apple          203.0    6            12           48
..           ...            ...  ...           ...          ...
925         poco          571.0    8             8            8
926         poco          571.0    8             8            8
927      Samsung          239.0   12            10           50
928      Samsung          239.0   12            10           50
929      Samsung          239.0   12            10           50

                Processor Battery Capacity  Screen Size    price  \
0             A17 Bionic             3600          6.1    79999
1             A17 Bionic             3600          6.1    84999
2             A17 Bionic             3600          6.1    89999
3             A17 Bionic             4200          6.7    89999
4             A17 Bionic             4200          6.7    94999
..                   ...              ...          ...      ...
925  Snapdragon 7s Gen 2            10000         12.1    23999
926  Snapdragon 7s Gen 2            10000         12.1    25999
927   Snapdragon 8 Gen 3             4400          7.6   164999
928   Snapdragon 8 Gen 3             4400          7.6   176999
929   Snapdragon 8 Gen 3             4400          7.6   200999

     Launched Year storage
0             2024     128
1             2024     256
2             2024     512
```

```
3              2024     128
4              2024     256
..              ...     ...
925            2024     128
926            2024     256
927            2024     256
928            2024     512
929            2024    1024

[930 rows x 11 columns]
```

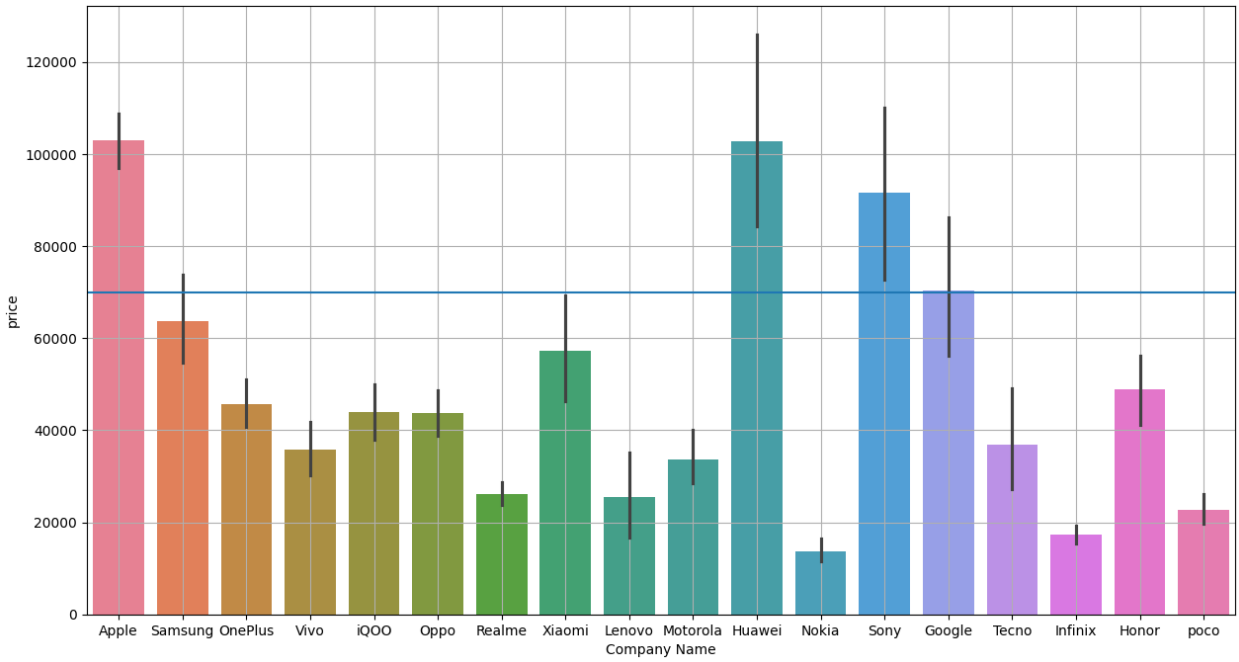
```
df["price"]=df["price"].astype(int)

sns.histplot(data=df["price"],bins=10,color="lightblue")
plt.grid()
plt.show()
```




```
#check if there are outlier in price
sns.boxplot(df["price"],color="lightblue")
plt.grid()
plt.show()
```




```
#different company distribution of price
plt.figure(figsize=(15,8))
sns.barplot(y=df["price"],x=df["Company Name"],hue=df["Company Name"])
plt.grid()
plt.axhline(70000)
plt.show()
```



```
df["Battery Capacity"]=df["Battery Capacity"].astype(int)

df
```

```
    Company Name  Mobile Weight  RAM  Front Camera  Back Camera  \
0          Apple          174.0    6            12           48
1          Apple          174.0    6            12           48
2          Apple          174.0    6            12           48
3          Apple          203.0    6            12           48
4          Apple          203.0    6            12           48
..           ...            ...  ...           ...          ...
925         poco          571.0    8             8            8
926         poco          571.0    8             8            8
927      Samsung          239.0   12            10           50
928      Samsung          239.0   12            10           50
929      Samsung          239.0   12            10           50

               Processor  Battery Capacity  Screen Size   price  \
0             A17 Bionic              3600          6.1   79999
1             A17 Bionic              3600          6.1   84999
2             A17 Bionic              3600          6.1   89999
3             A17 Bionic              4200          6.7   89999
4             A17 Bionic              4200          6.7   94999
..                   ...               ...          ...     ...
925  Snapdragon 7s Gen 2             10000         12.1   23999
926  Snapdragon 7s Gen 2             10000         12.1   25999
927   Snapdragon 8 Gen 3              4400          7.6  164999
928   Snapdragon 8 Gen 3              4400          7.6  176999
929   Snapdragon 8 Gen 3              4400          7.6  200999

     Launched Year storage
0             2024     128
1             2024     256
2             2024     512
3             2024     128
4             2024     256
..             ...     ...
925           2024     128
926           2024     256
927           2024     256
928           2024     512
929           2024    1024

[930 rows x 11 columns]
```

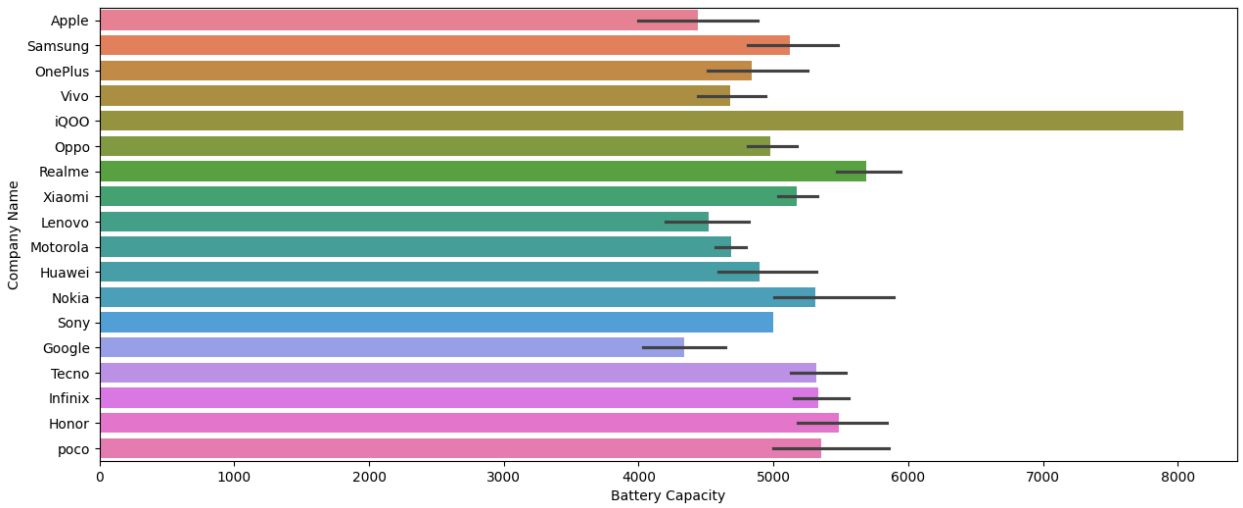
```
df["RAM"]=df["RAM"].astype(int)

#correlation between all numerical value
dat_corr=df.drop(columns=["Company Name","Processor"])

dat_corr.corr()
```

```
                    Mobile Weight       RAM  Front Camera  Back
Camera  \
Mobile Weight           1.000000 -0.008053     -0.282998    -0.308455

RAM                    -0.008053  1.000000      0.459337     0.439794

Front Camera           -0.282998  0.459337      1.000000     0.449008

Back Camera            -0.308455  0.439794      0.449008     1.000000

Battery Capacity        0.847731  0.139123     -0.159349    -0.105918

Screen Size             0.975893  0.046496     -0.224008    -0.263217

price                   0.141899  0.422192      0.022681     0.035269

Launched Year           0.066349  0.381142      0.165645     0.318243

storage                 0.025781  0.365012      0.196915     0.069495


                    Battery Capacity  Screen Size     price  Launched
Year  \
Mobile Weight              0.847731     0.975893  0.141899
0.066349
RAM                        0.139123     0.046496  0.422192
0.381142
Front Camera              -0.159349    -0.224008  0.022681
0.165645
Back Camera               -0.105918    -0.263217  0.035269
0.318243
Battery Capacity           1.000000     0.879453 -0.007829
0.351973
Screen Size                0.879453     1.000000  0.108592
0.139199
price                     -0.007829     0.108592  1.000000
0.070470
Launched Year              0.351973     0.139199  0.070470
1.000000
storage                    0.046456     0.013568  0.404761
0.143824

                     storage
Mobile Weight       0.025781
RAM                 0.365012
Front Camera        0.196915
Back Camera         0.069495
Battery Capacity    0.046456
Screen Size         0.013568
price               0.404761
```

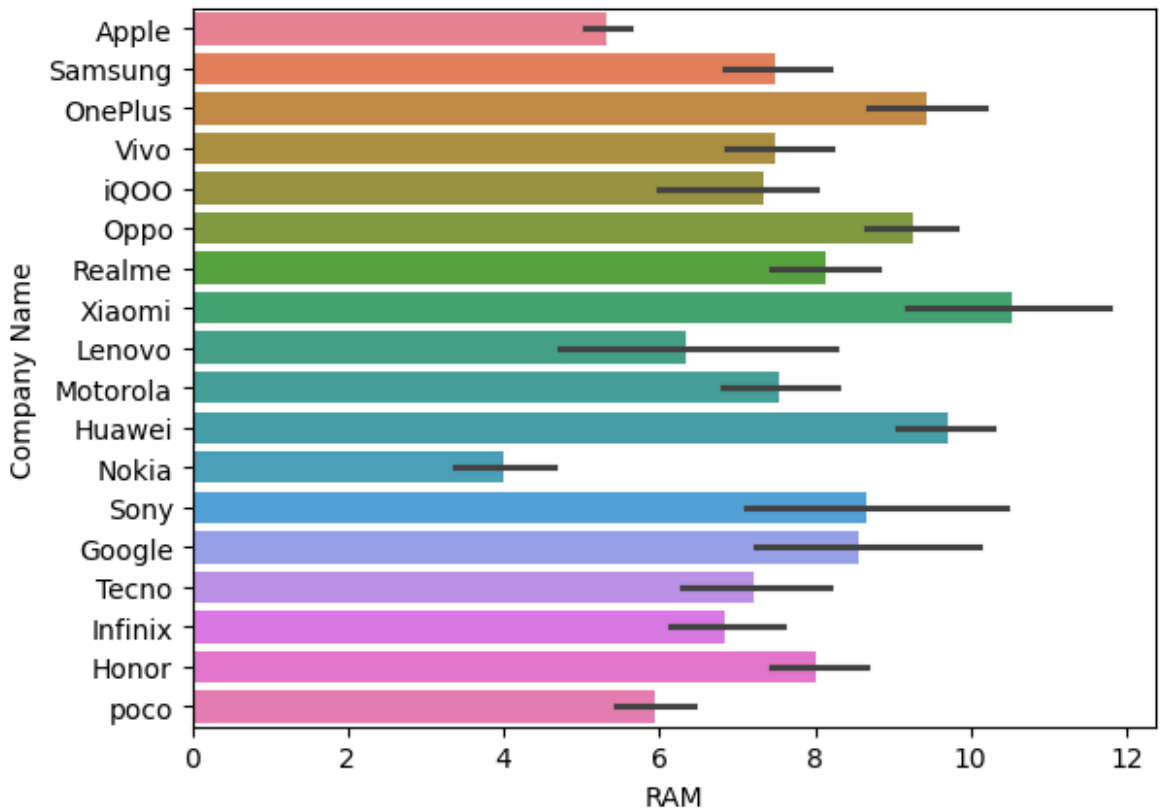
```
Launched Year     0.143824
storage           1.000000
```

```
#distribution of battery capacity
plt.figure(figsize=(15,6))
sns.barplot(x=df["Battery Capacity"],y=df["Company
Name"],hue=df["Company Name"])
```

```
<Axes: xlabel='Battery Capacity', ylabel='Company Name'>
```

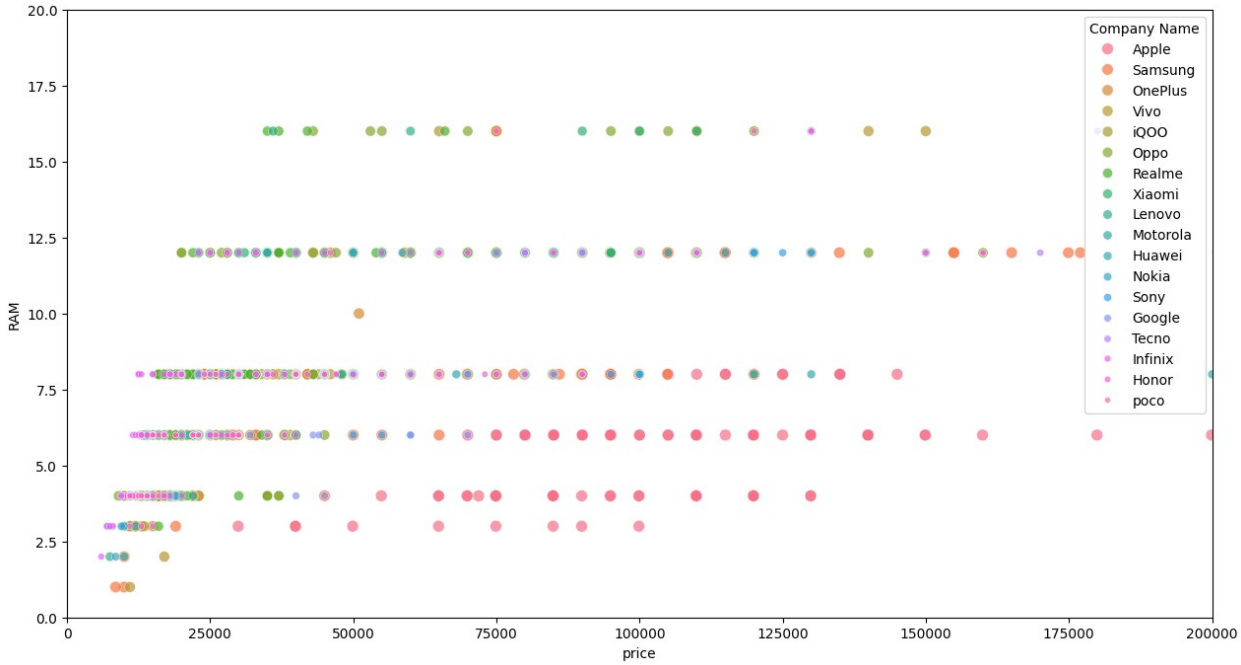



```
#ram distribution with company
sns.barplot(x=df["RAM"],y=df["Company Name"],hue=df["Company Name"])

plt.show()
```



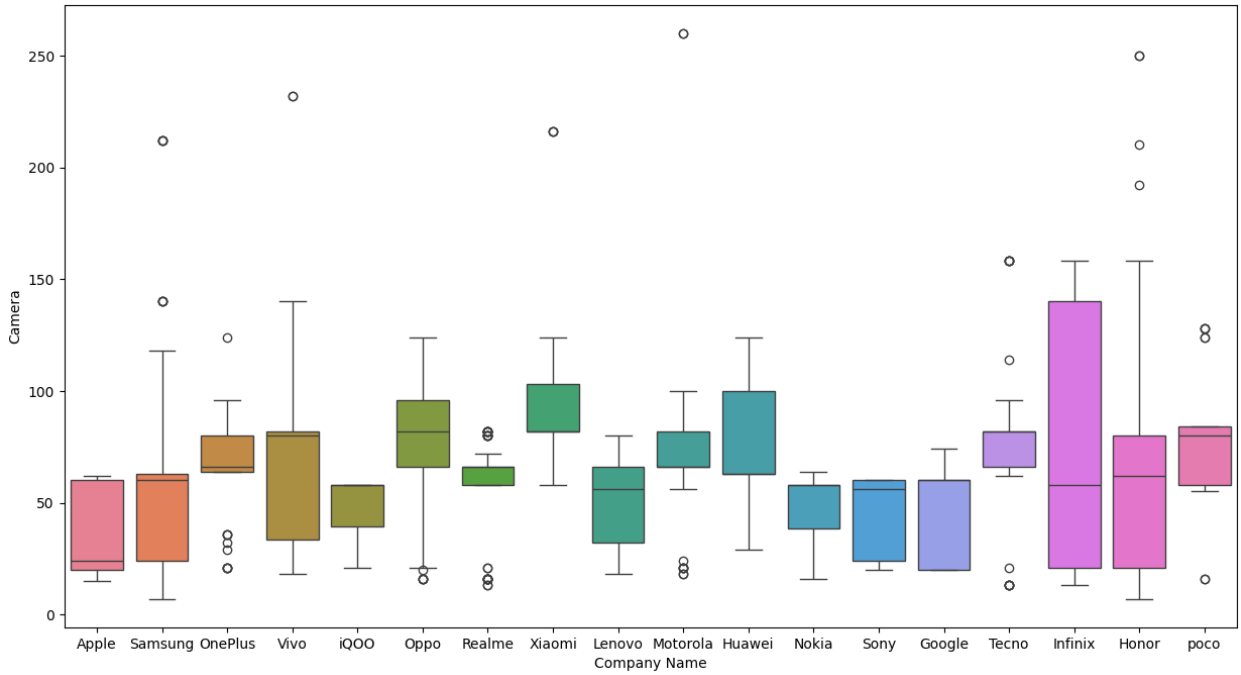

```
#checking price v/s ram
plt.figure(figsize=(15,8))
sns.scatterplot(x=df["price"],y=df["RAM"],hue=df["Company
Name"],size=df["Company Name"],alpha=0.7)
plt.xlim(0,200000)
plt.ylim(0,20)
```

```
(0.0, 20.0)
```



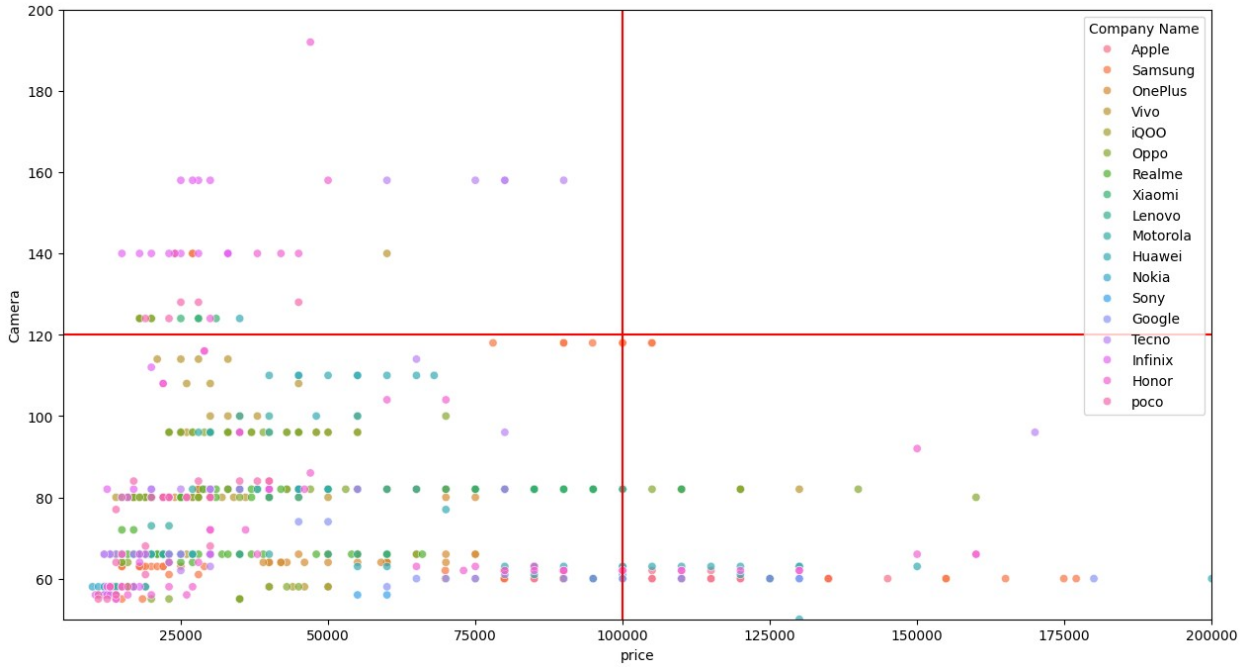

```
#camera quality v/s price
#first we need to combine front and back camera to show full megapixel
df["Camera"]=df["Front Camera"]+df["Back Camera"]

plt.figure(figsize=(15,8))
sns.boxplot(x=df["Company Name"],y=df["Camera"],hue=df["Company
Name"])
```

```
<Axes: xlabel='Company Name', ylabel='Camera'>
```

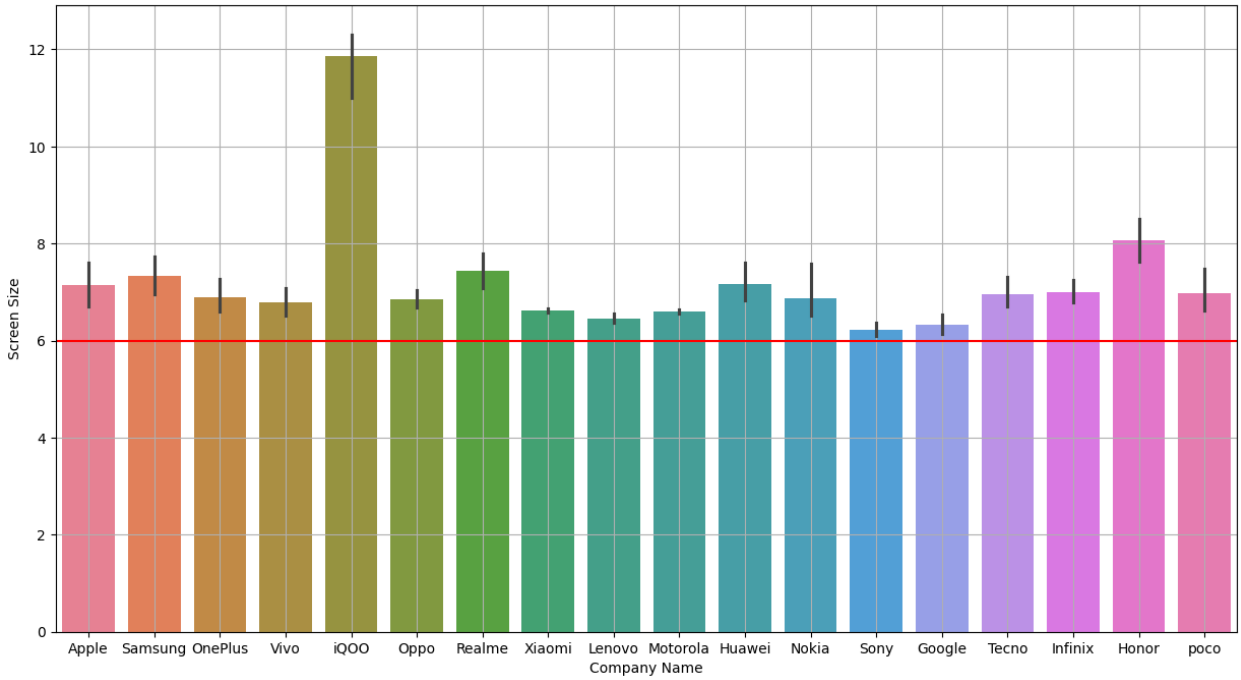



```
#price v/s camera quality
plt.figure(figsize=(15,8))
sns.scatterplot(x=df["price"],y=df["Camera"],hue=df["Company
Name"],alpha=0.7)
plt.xlim(5000,200000)
plt.ylim(50,200)
plt.axhline(120,color="red")
plt.axvline(100000,color="red")
```

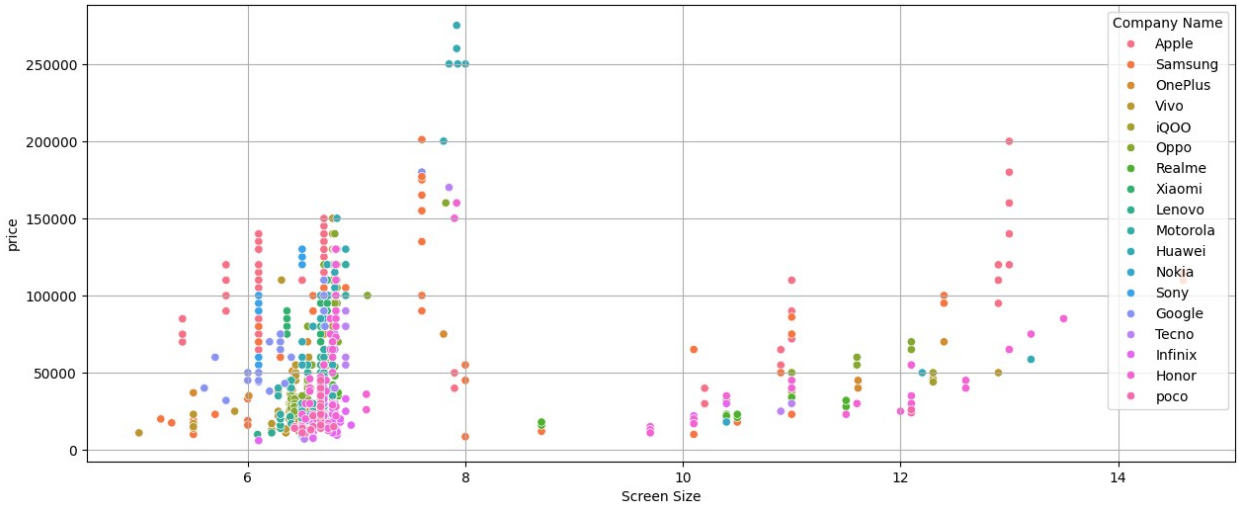
```
<matplotlib.lines.Line2D at 0x21ee2380470>
```



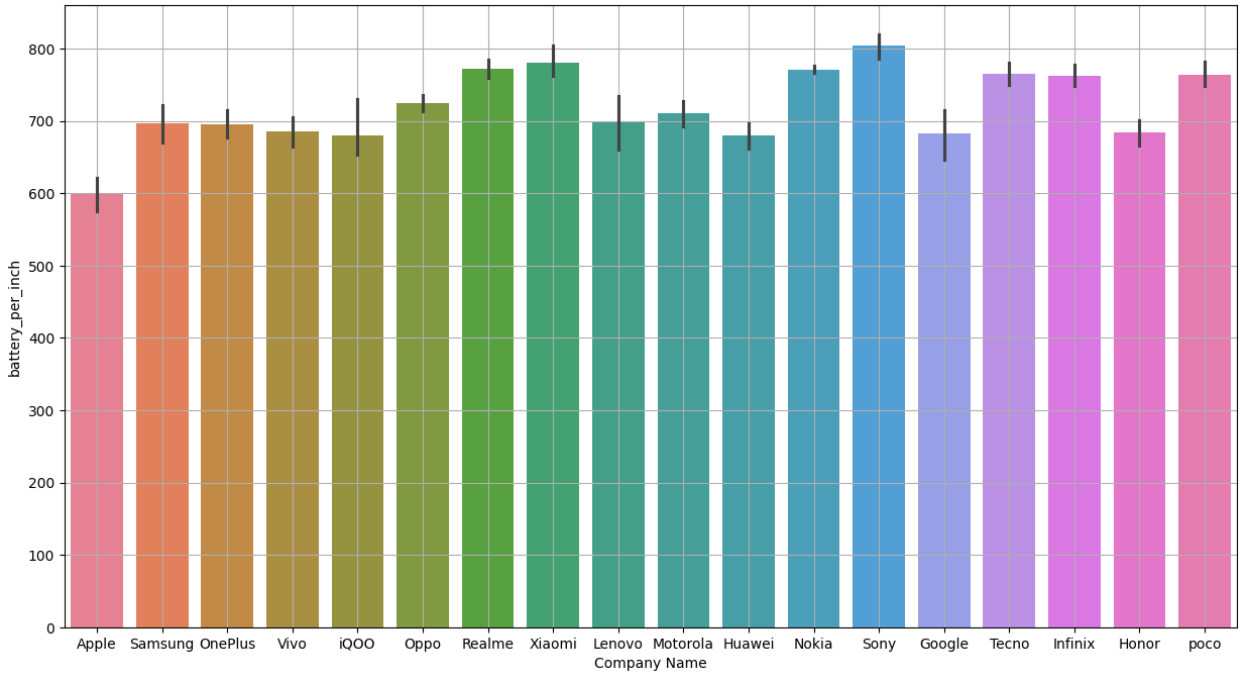

```
#distribution of screen size and company
plt.figure(figsize=(15,8))
sns.barplot(y=df["Screen Size"],x=df["Company Name"],hue=df["Company Name"])
plt.axhline(6,color="red")
plt.grid()
```



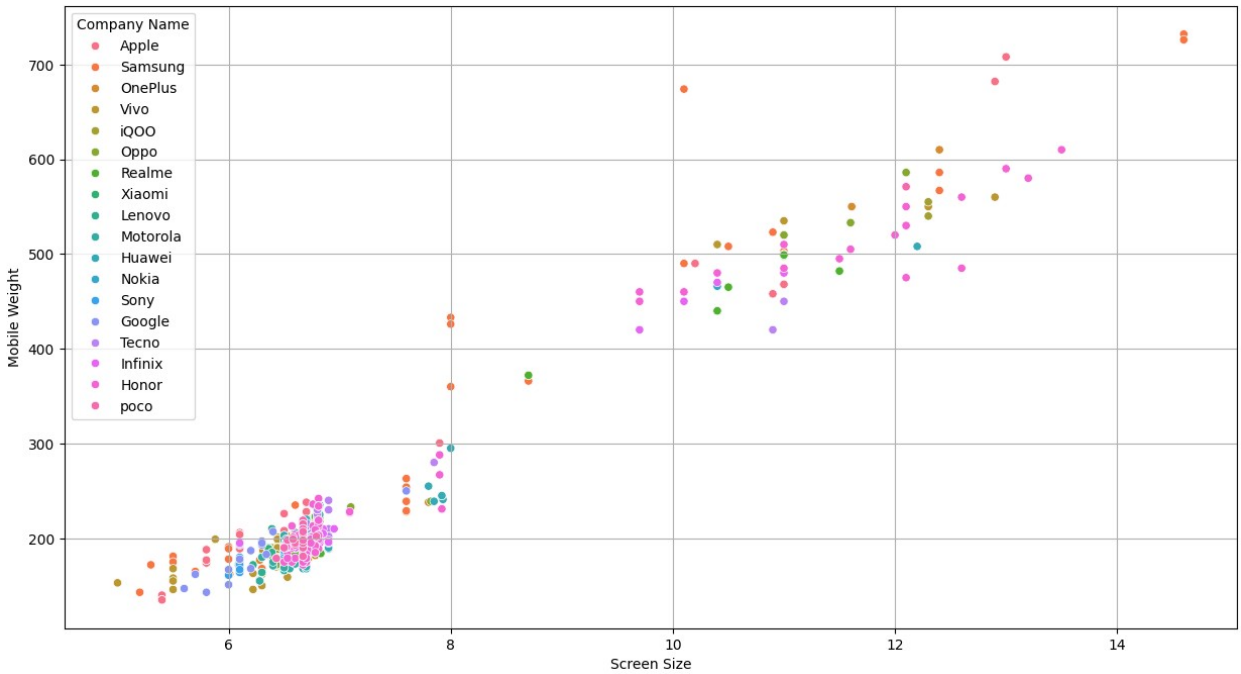
```
#screen size v/s price
plt.figure(figsize=(15,6))
sns.scatterplot(x=df["Screen Size"],y=df["price"],hue=df["Company
Name"])
plt.grid()
```




```
#battery per inch
df["battery_per_inch"]=df["Battery Capacity"]/df["Screen Size"]

plt.figure(figsize=(15,8))
sns.barplot(y=df["battery_per_inch"],x=df["Company
Name"],hue=df["Company Name"])
plt.grid()
```



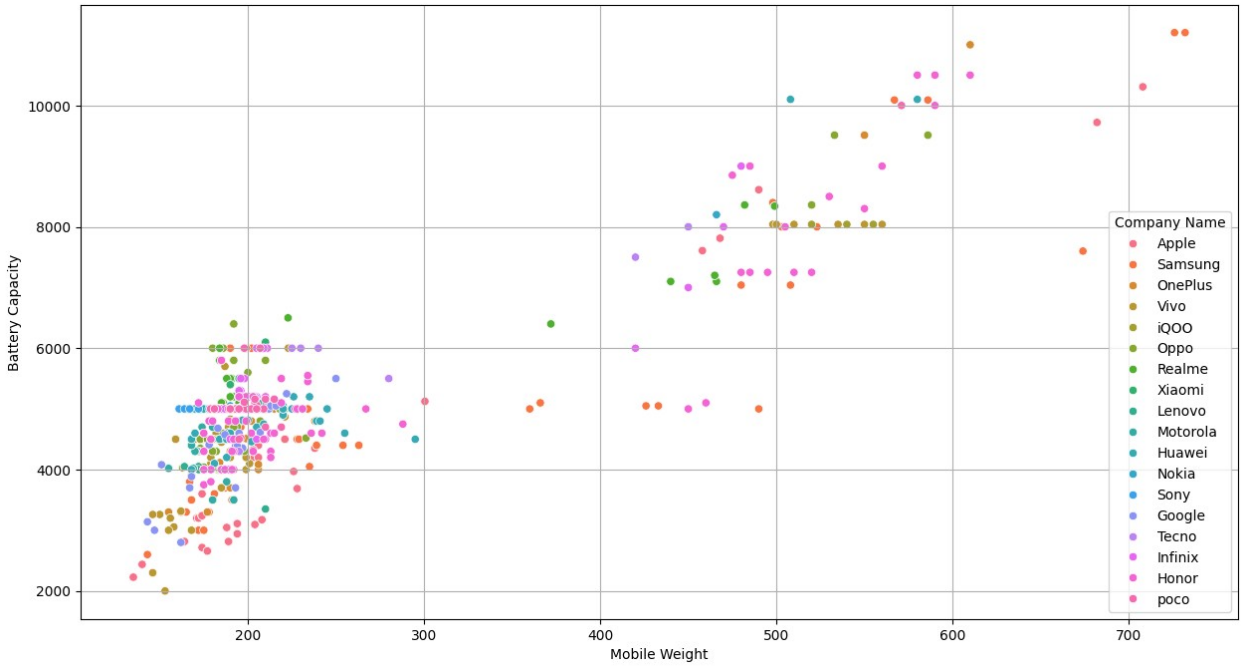

```
#how does mobile weight relate to screen size and battery capacity
across different companies
plt.figure(figsize=(15,8))
sns.scatterplot(x=df["Screen Size"],y=df["Mobile
Weight"],hue=df["Company Name"])
plt.grid()
```



```
df
```

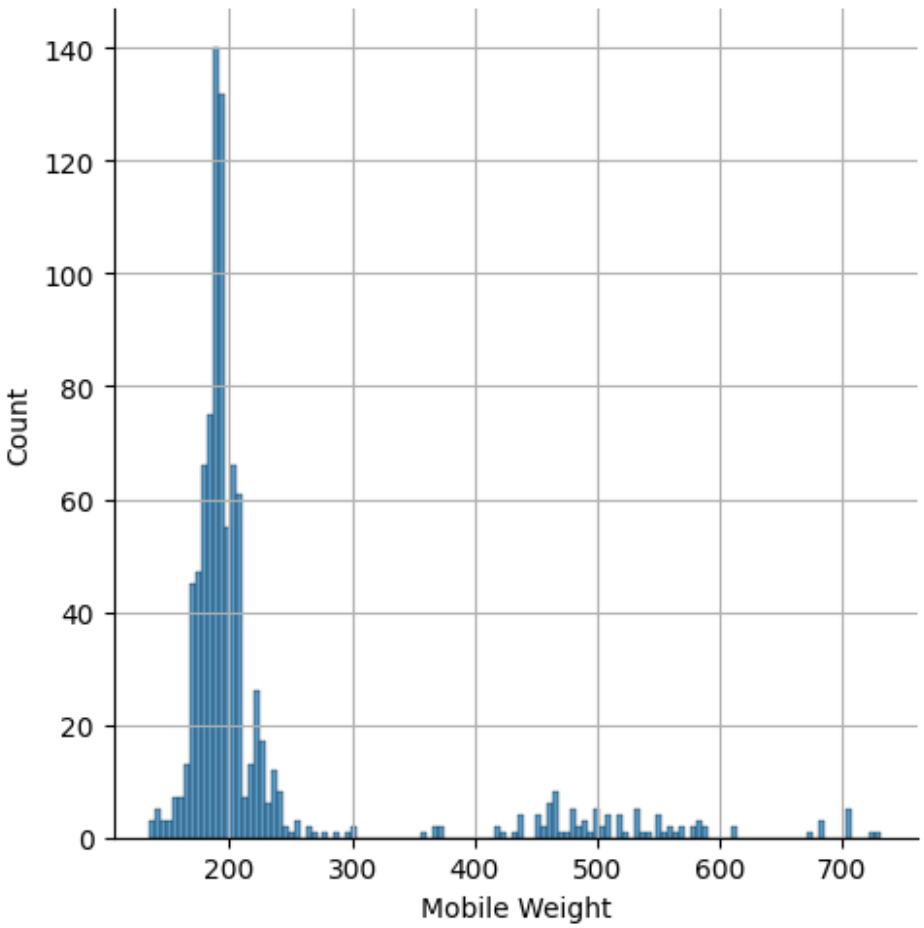
```
    Company Name  Mobile Weight  RAM  Front Camera  Back Camera  \
0          Apple          174.0    6            12           48
1          Apple          174.0    6            12           48
2          Apple          174.0    6            12           48
3          Apple          203.0    6            12           48
4          Apple          203.0    6            12           48
..           ...            ...  ...           ...          ...
925         poco          571.0    8             8            8
926         poco          571.0    8             8            8
927      Samsung          239.0   12            10           50
928      Samsung          239.0   12            10           50
929      Samsung          239.0   12            10           50

               Processor  Battery Capacity  Screen Size   price  \
0             A17 Bionic              3600          6.1   79999
1             A17 Bionic              3600          6.1   84999
2             A17 Bionic              3600          6.1   89999
3             A17 Bionic              4200          6.7   89999
4             A17 Bionic              4200          6.7   94999
..                   ...               ...          ...     ...
925  Snapdragon 7s Gen 2             10000         12.1   23999
926  Snapdragon 7s Gen 2             10000         12.1   25999
927   Snapdragon 8 Gen 3              4400          7.6  164999
928   Snapdragon 8 Gen 3              4400          7.6  176999
929   Snapdragon 8 Gen 3              4400          7.6  200999

     Launched Year storage  Camera  battery_per_inch
0             2024     128      60        590.163934
1             2024     256      60        590.163934
2             2024     512      60        590.163934
3             2024     128      60        626.865672
4             2024     256      60        626.865672
..             ...     ...     ...               ...
925           2024     128      16        826.446281
926           2024     256      16        826.446281
927           2024     256      60        578.947368
928           2024     512      60        578.947368
929           2024    1024      60        578.947368

[930 rows x 13 columns]
```

```
#battery v/s weight
plt.figure(figsize=(15,8))
sns.scatterplot(y=df["Battery Capacity"],x=df["Mobile
Weight"],hue=df["Company Name"])
plt.grid()
```



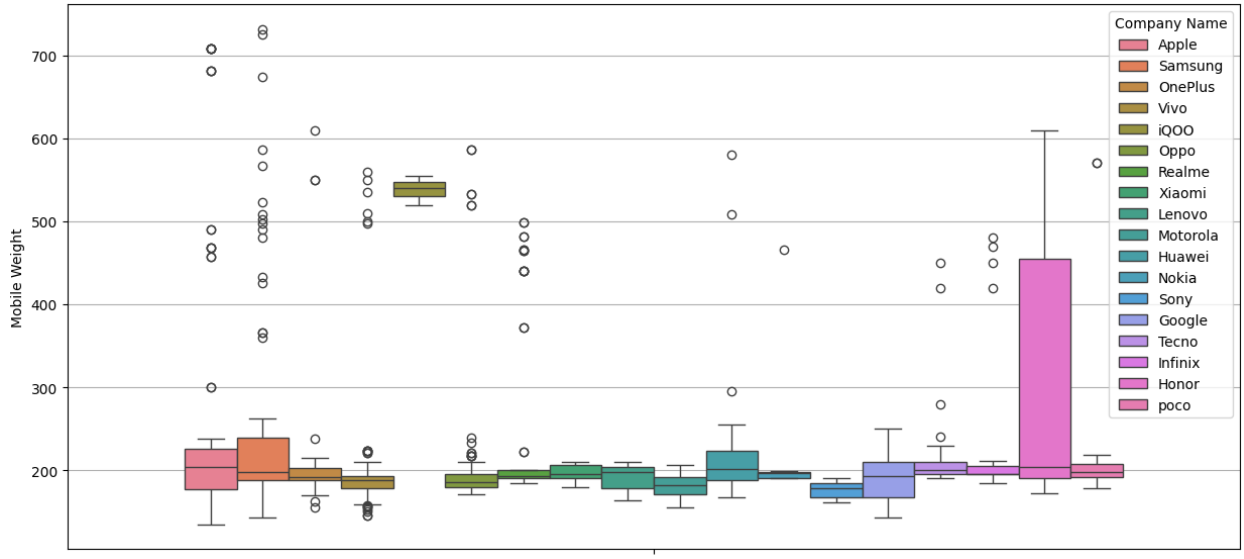

```
#density of weight
plt.figure(figsize=(15,8))
sns.displot(df["Mobile Weight"])
plt.grid()
plt.show()
```

```
<Figure size 1500x800 with 0 Axes>
```




```
#weight distribution by company
plt.figure(figsize=(15,7))
plt.grid()
sns.boxplot(y=df["Mobile Weight"],hue=df["Company Name"])
```

```
<Axes: ylabel='Mobile Weight'>
```

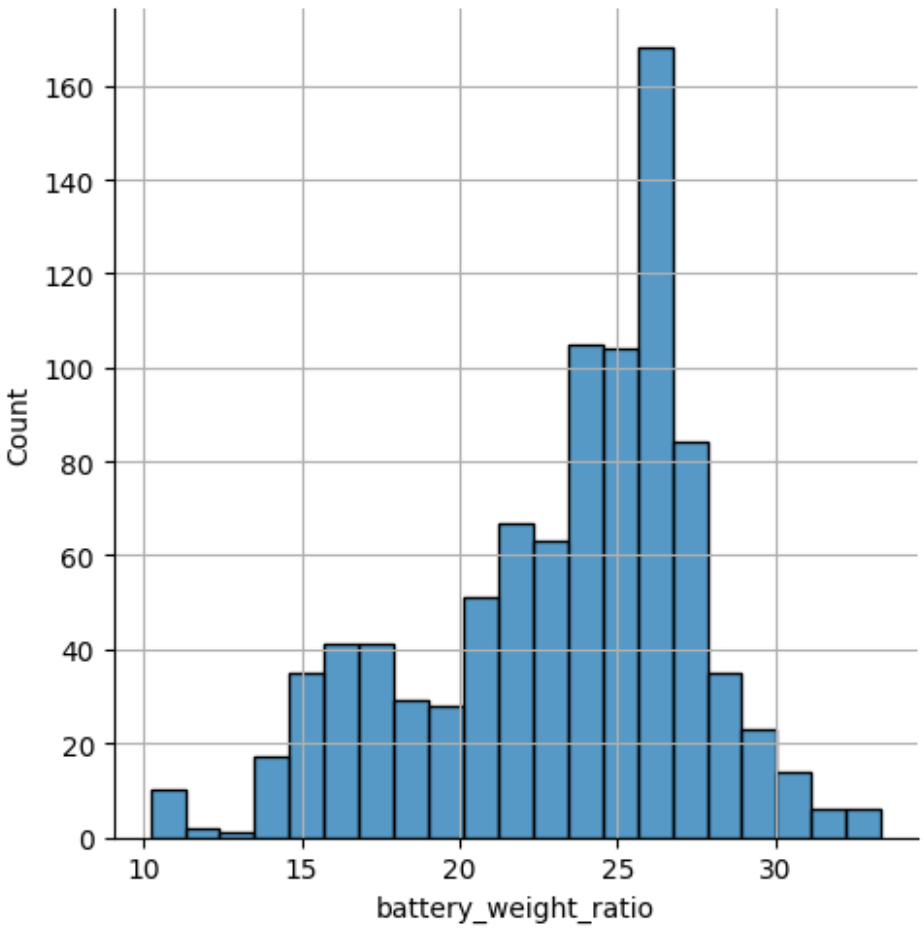



```
#battery to weight ratio
df["battery_weight_ratio"]=df["Battery Capacity"]/df["Mobile Weight"]

df
```

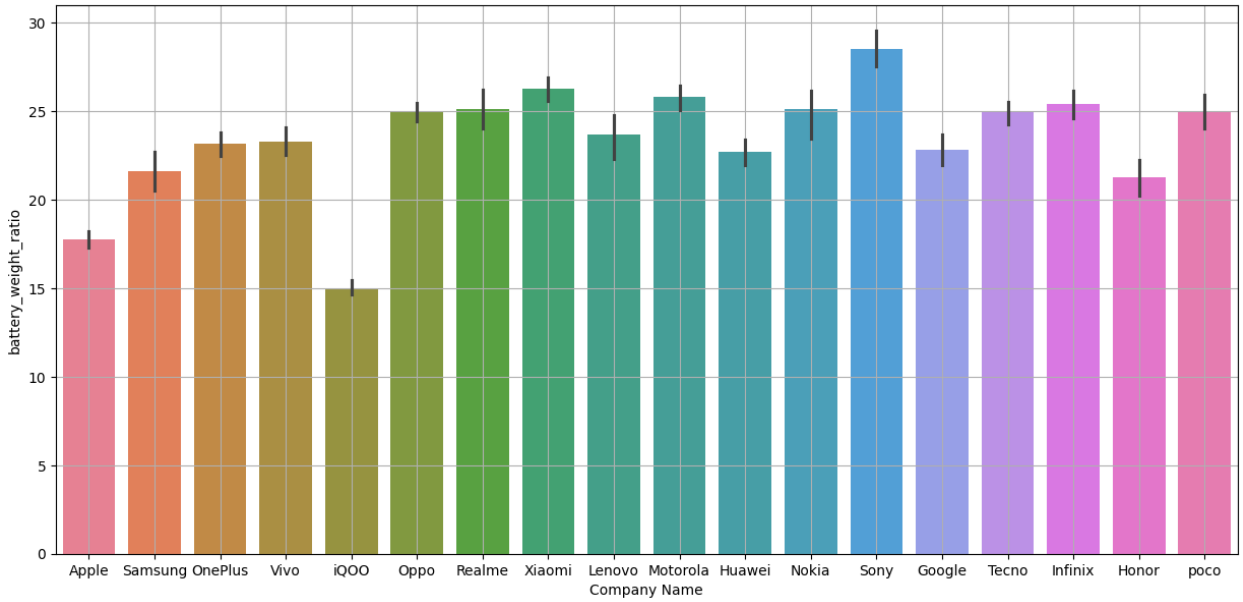
```
    Company Name  Mobile Weight  RAM  Front Camera  Back Camera  \
0          Apple          174.0    6            12           48
1          Apple          174.0    6            12           48
2          Apple          174.0    6            12           48
3          Apple          203.0    6            12           48
4          Apple          203.0    6            12           48
..           ...            ...  ...           ...          ...
925         poco          571.0    8             8            8
926         poco          571.0    8             8            8
927      Samsung          239.0   12            10           50
928      Samsung          239.0   12            10           50
929      Samsung          239.0   12            10           50

                Processor  Battery Capacity  Screen Size   price  \
0             A17 Bionic              3600          6.1   79999
1             A17 Bionic              3600          6.1   84999
2             A17 Bionic              3600          6.1   89999
3             A17 Bionic              4200          6.7   89999
4             A17 Bionic              4200          6.7   94999
..                   ...               ...          ...     ...
925  Snapdragon 7s Gen 2             10000         12.1   23999
926  Snapdragon 7s Gen 2             10000         12.1   25999
927   Snapdragon 8 Gen 3              4400          7.6  164999
928   Snapdragon 8 Gen 3              4400          7.6  176999
929   Snapdragon 8 Gen 3              4400          7.6  200999

     Launched Year storage  Camera  battery_per_inch
battery_weight_ratio
```

```
0              2024     128      60         590.163934
20.689655
1              2024     256      60         590.163934
20.689655
2              2024     512      60         590.163934
20.689655
3              2024     128      60         626.865672
20.689655
4              2024     256      60         626.865672
20.689655
..              ...     ...     ...                ...
...
925            2024     128      16         826.446281
17.513135
926            2024     256      16         826.446281
17.513135
927            2024     256      60         578.947368
18.410042
928            2024     512      60         578.947368
18.410042
929            2024    1024      60         578.947368
18.410042

[930 rows x 14 columns]
```

```
#where battery_weight_ratio id dense
sns.displot(df["battery_weight_ratio"])
plt.grid()
plt.show()
```




```
#battery weight ratio for different company
plt.figure(figsize=(15,7))
sns.barplot(y=df["battery_weight_ratio"],x=df["Company
Name"],hue=df["Company Name"])
plt.grid()
plt.show()
```

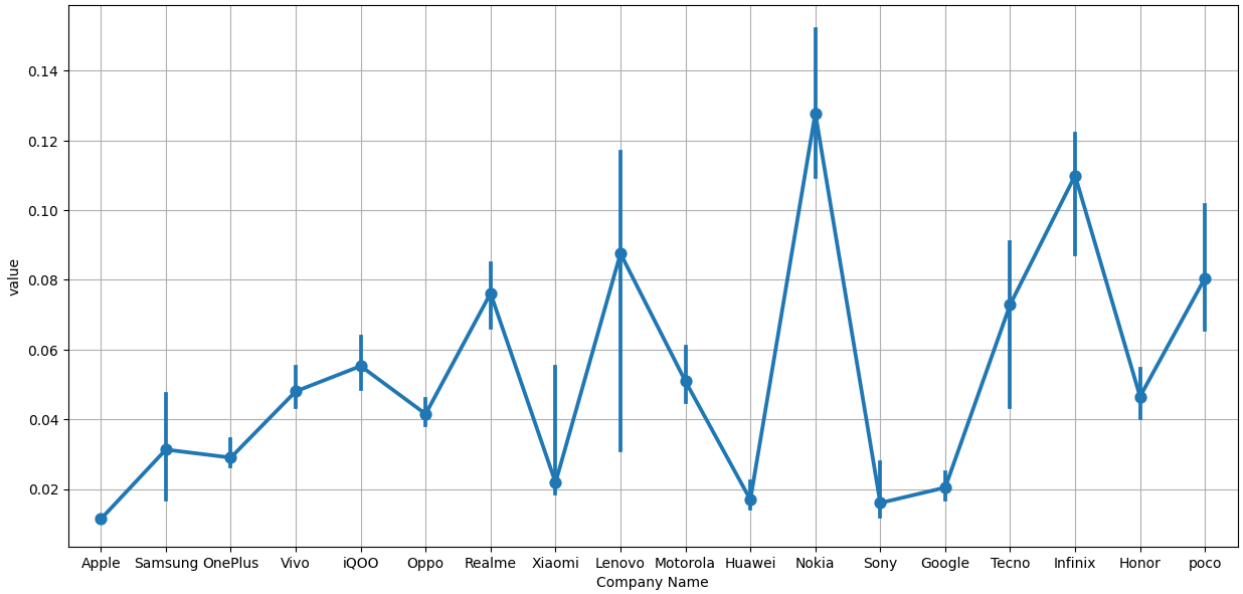



```
#which company provide the best value for money when considering
ram,battery capacity,camera and price together
#for this we must take ratio of ram,battery capacity and camera
#camera is most needed so camera weight would be 0.4, battery would be
0.3, ram also would be 0.3
#not include price as it would increase the performance means higher
cost=higher performance so we would not include it
df["performance"]=(0.4*df["RAM"])+(0.3*df["Camera"])+(0.3*df["Battery
Capacity"])
#after this we would divide it with price
df["value"]=df["performance"]/df["price"]

df
```

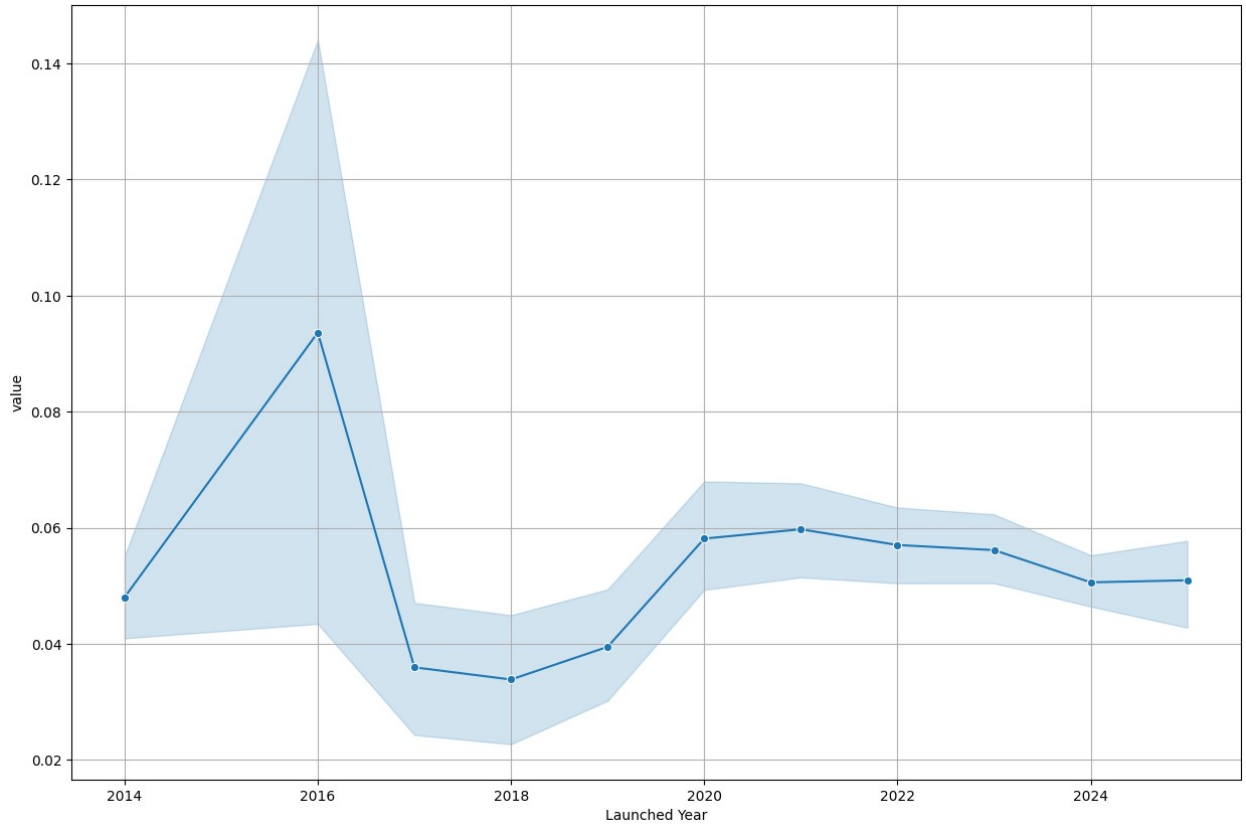
```
    Company Name  Mobile Weight  RAM  Front Camera  Back Camera  \
0          Apple          174.0    6            12           48
1          Apple          174.0    6            12           48
2          Apple          174.0    6            12           48
3          Apple          203.0    6            12           48
4          Apple          203.0    6            12           48
..           ...            ...  ...           ...          ...
925         poco          571.0    8             8            8
926         poco          571.0    8             8            8
927      Samsung          239.0   12            10           50
928      Samsung          239.0   12            10           50
929      Samsung          239.0   12            10           50

                Processor  Battery Capacity  Screen Size   price  \
0              A17 Bionic              3600          6.1   79999
1              A17 Bionic              3600          6.1   84999
2              A17 Bionic              3600          6.1   89999
```

```
3             A17 Bionic              4200          6.7   89999
4             A17 Bionic              4200          6.7   94999
..                   ...               ...          ...     ...
925  Snapdragon 7s Gen 2             10000         12.1   23999
926  Snapdragon 7s Gen 2             10000         12.1   25999
927   Snapdragon 8 Gen 3              4400          7.6  164999
928   Snapdragon 8 Gen 3              4400          7.6  176999
929   Snapdragon 8 Gen 3              4400          7.6  200999

     Launched Year storage  Camera  battery_per_inch
battery_weight_ratio  \
0             2024     128      60        590.163934
20.689655
1             2024     256      60        590.163934
20.689655
2             2024     512      60        590.163934
20.689655
3             2024     128      60        626.865672
20.689655
4             2024     256      60        626.865672
20.689655
..             ...     ...     ...               ...
...
925           2024     128      16        826.446281
17.513135
926           2024     256      16        826.446281
17.513135
927           2024     256      60        578.947368
18.410042
928           2024     512      60        578.947368
18.410042
929           2024    1024      60        578.947368
18.410042

     performance     value
0         1100.4  0.013755
1         1100.4  0.012946
2         1100.4  0.012227
3         1280.4  0.014227
4         1280.4  0.013478
..           ...       ...
925       3008.0  0.125339
926       3008.0  0.115697
927       1342.8  0.008138
928       1342.8  0.007586
929       1342.8  0.006681

[930 rows x 16 columns]
```

```
plt.figure(figsize=(15,7))
sns.pointplot(y=df["value"],x=df["Company Name"],estimator="median")
plt.grid()
plt.show()
```



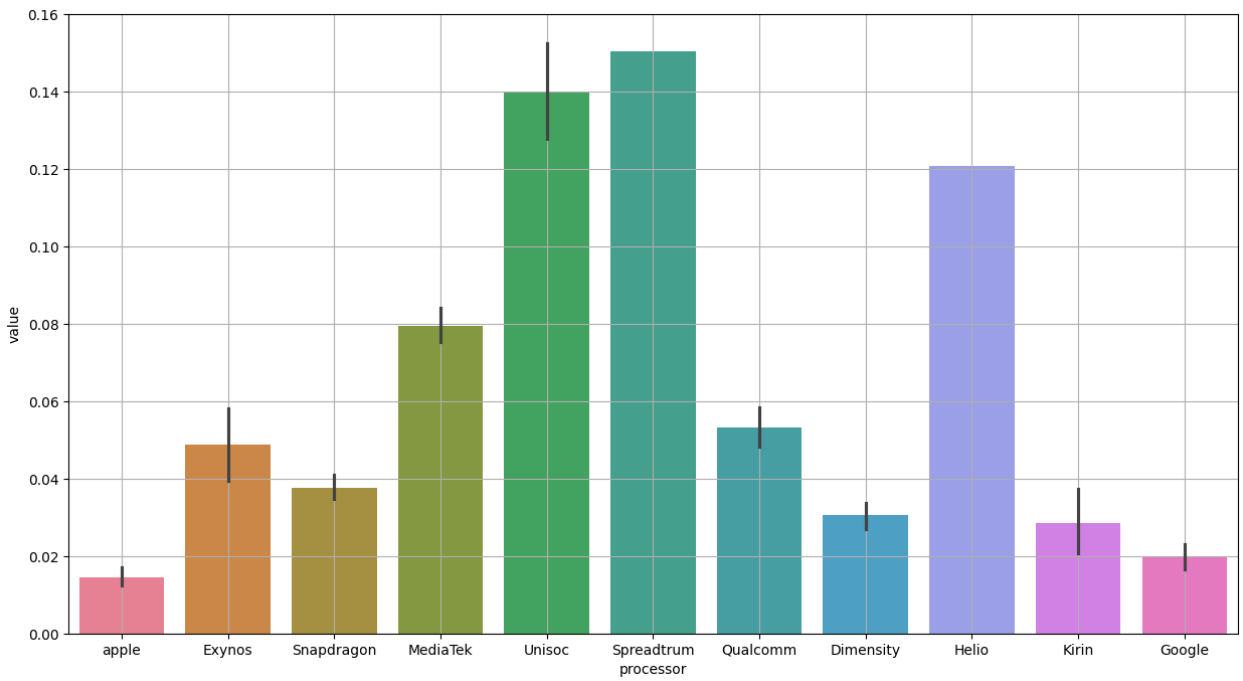

```
#do newer phones actually provide better value for money compared to
older launches
plt.figure(figsize=(15,10))
sns.lineplot(y=df["value"],x=df["Launched Year"],marker="o")
plt.grid()
plt.show()
```




```python
#does processor type influence value for money or it mainly driven by other specification
#for this we would need to extract meaningfull part from the data
df["processor"]=df["Processor"].str.extract(r'^(\w+)')

df["apple"]=df["processor"].str.extract(r'^(A)')

df["Processor"].str.extract(r'^(\w+)')
```

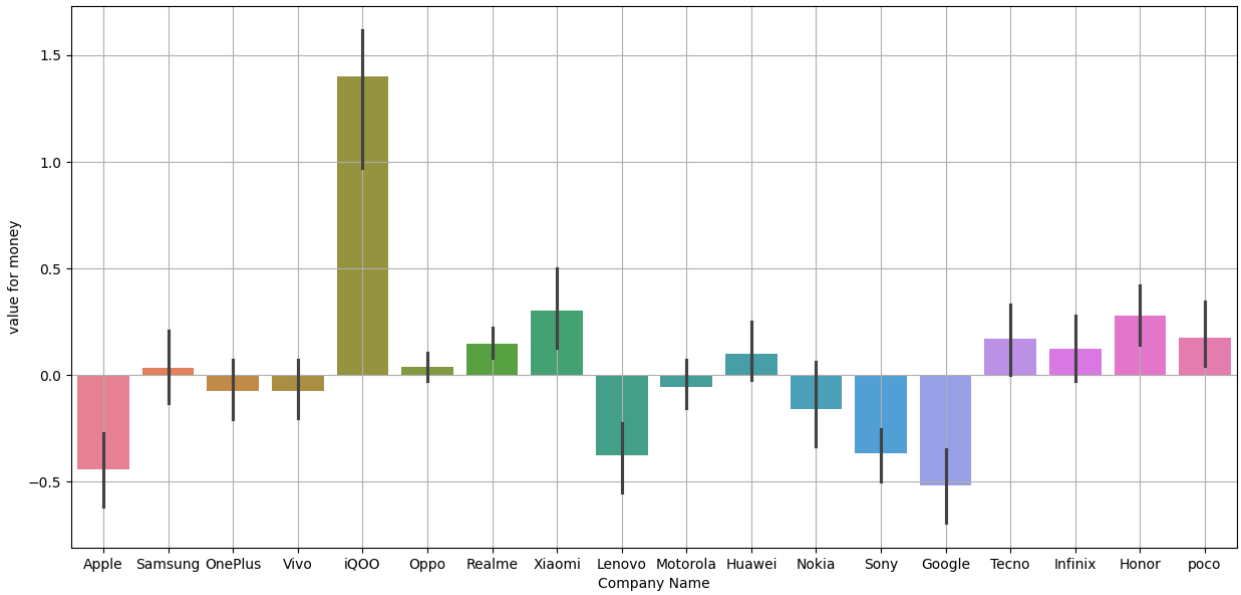
```
               0
0           A17
1           A17
2           A17
3           A17
4           A17
..          ...
925  Snapdragon
926  Snapdragon
927  Snapdragon
928  Snapdragon
929  Snapdragon

[930 rows x 1 columns]
```

```
#replacinf a17,a16 as a
df["processor"]=df["apple"].combine_first(df["processor"])

#drop apple column
df.drop(columns=["apple"],inplace=True)

#now replace a as apple
df["processor"]=df["processor"].replace({"A":"apple"})

#now lets visualize if processor influence the value for money or it
dpeend on other specification
plt.figure(figsize=(15,8))
sns.barplot(x=df["processor"],y=df["value"],hue=df["processor"])
plt.grid()
plt.show()
```

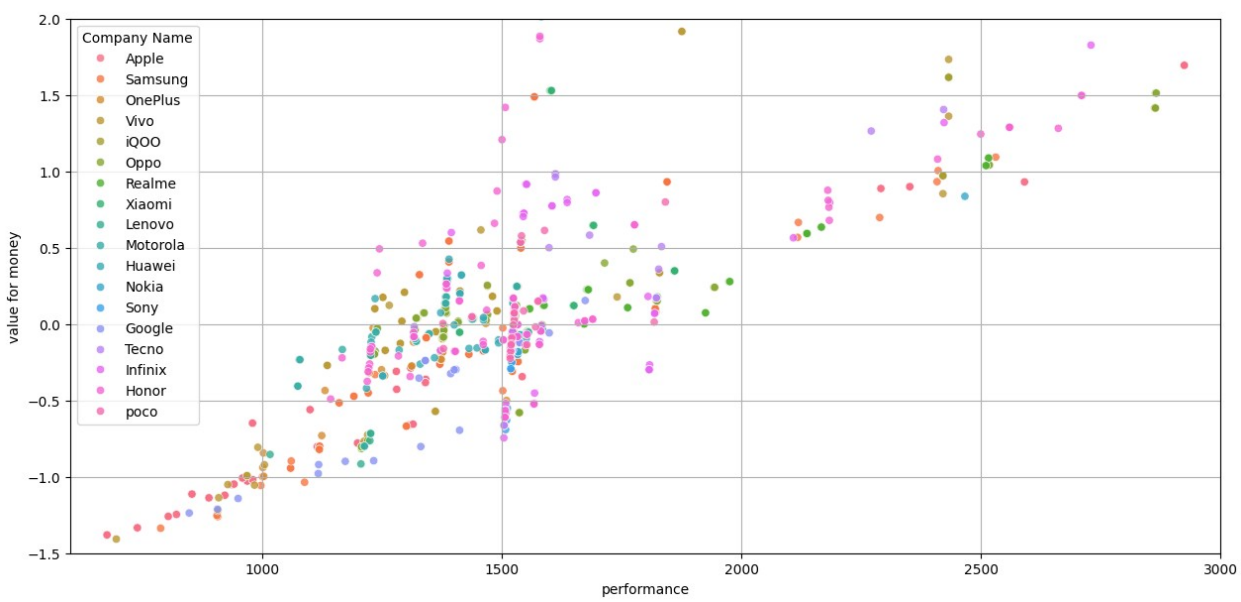



```
#are value for money processor compromising on other critical aspects
like camera battery capacityor display quality
#before weighting them we need to normalize them
from sklearn.preprocessing import StandardScaler
scaler=StandardScaler()
df[["camera_s","battery_s","display_a"]]=scaler.fit_transform(df[["Cam
era","Battery Capacity","Screen Size"]])

df["value for money"]=(0.4*df["camera_s"])+(0.3*df["battery_s"])
+(0.3*df["display_a"])

plt.figure(figsize=(15,7))
sns.barplot(x=df["Company Name"],y=df["value for
```

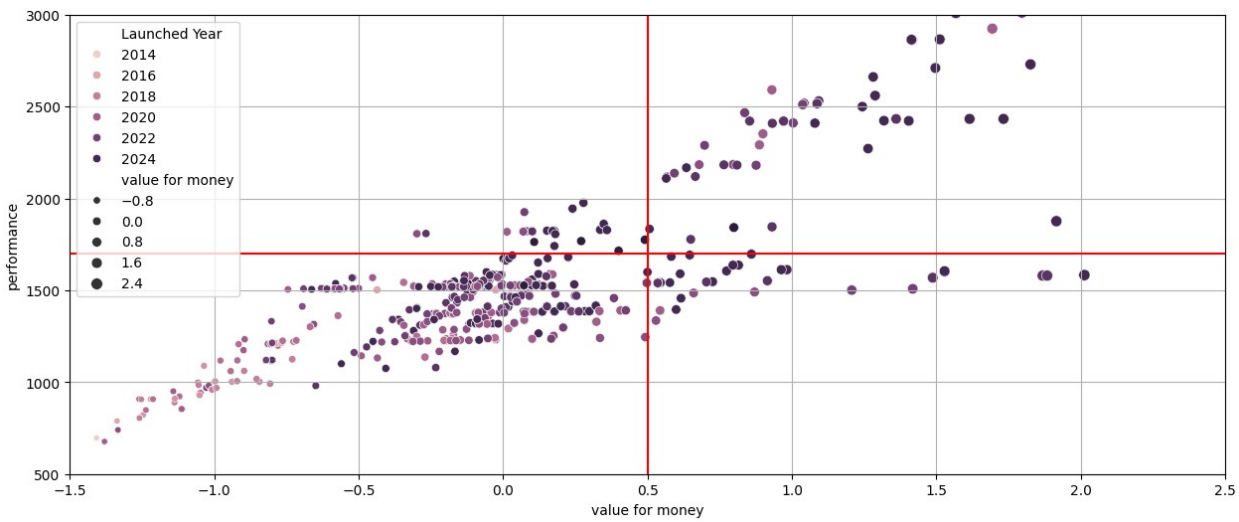
```
money"],hue=df["Company Name"])
plt.grid()
```




```
#which company provide best value for money and performence
plt.figure(figsize=(15,7))
sns.scatterplot(x=df["performance"],y=df["value for
money"],hue=df["Company Name"],alpha=0.8)
plt.xlim(600,3000)
plt.ylim(-1.5,2.0)
plt.grid()
plt.show()
```



```
#are newer phone providing better performence and value for money than
older phone
plt.figure(figsize=(15,6))
sns.scatterplot(x=df["value for
money"],y=df["performance"],hue=df["Launched Year"],size=df["value for
money"])
plt.xlim(-1.5,2.5)
plt.ylim(500,3000)
plt.axvline(0.5,color="red")
plt.axhline(1700,color="red")
plt.grid()
plt.show()
```

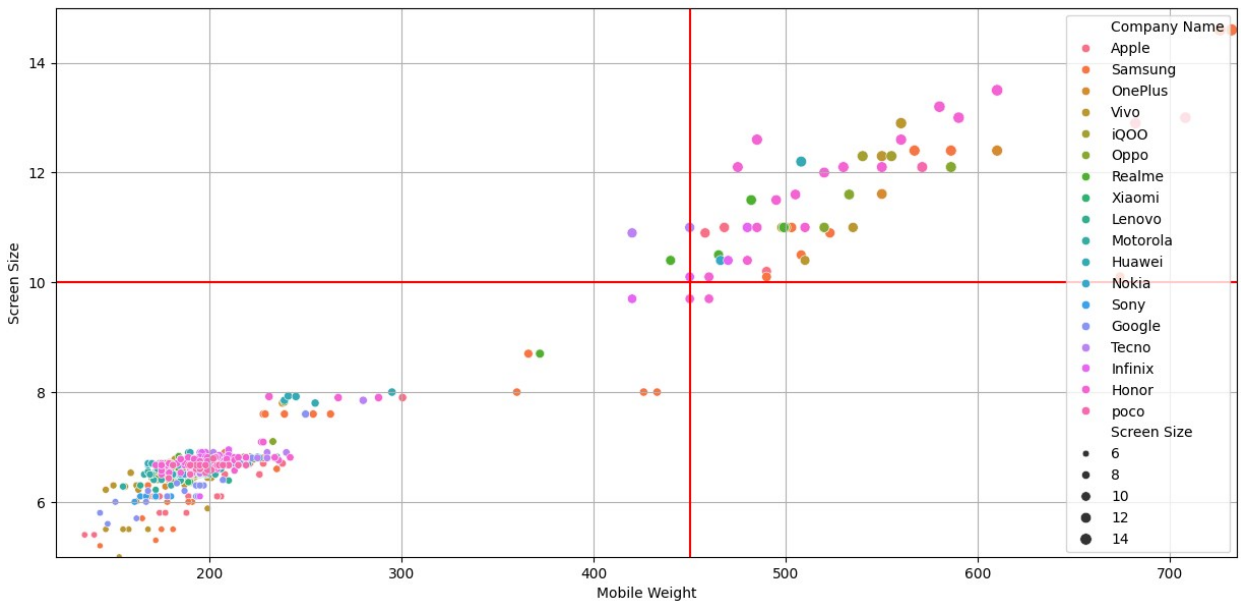



```
#which factor affect price of mobile the most
data_num=df.drop(columns=["Company Name","Processor","Launched
Year","battery_per_inch","battery_weight_ratio","processor","camera_s"
,"battery_s","value for money","display_a"])

corr=data_num.corr()
corr
```

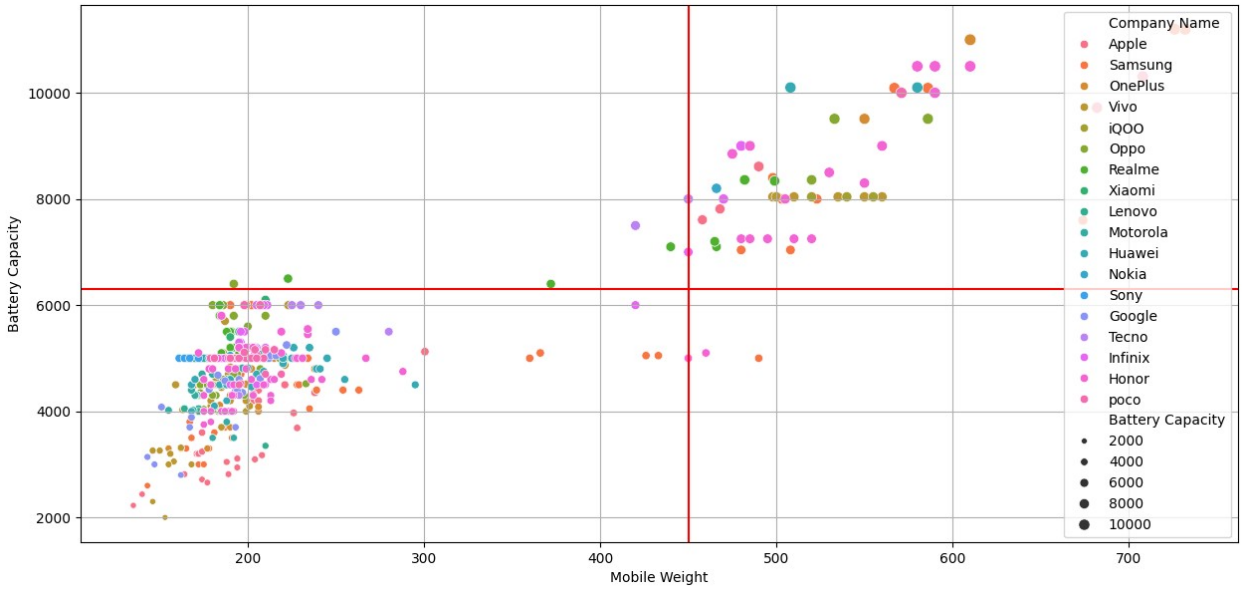
```
                   Mobile Weight       RAM  Front Camera  Back
Camera  \
Mobile Weight           1.000000 -0.008053     -0.282998    -0.308455

RAM                    -0.008053  1.000000      0.459337     0.439794

Front Camera           -0.282998  0.459337      1.000000     0.449008

Back Camera            -0.308455  0.439794      0.449008     1.000000

Battery Capacity        0.847731  0.139123     -0.159349    -0.105918
```

```
Screen Size            0.975893  0.046496     -0.224008    -0.263217

price                  0.141899  0.422192      0.022681     0.035269

storage                0.025781  0.365012      0.196915     0.069495

Camera                -0.341540  0.504589      0.682754     0.959416

performance            0.840624  0.156862     -0.139185    -0.077878

value                  0.054706 -0.528213     -0.324225    -0.193357


                   Battery Capacity  Screen Size     price   storage
Camera  \
Mobile Weight             0.847731     0.975893  0.141899  0.025781 -
0.341540
RAM                       0.139123     0.046496  0.422192  0.365012
0.504589
Front Camera             -0.159349    -0.224008  0.022681  0.196915
0.682754
Back Camera              -0.105918    -0.263217  0.035269  0.069495
0.959416
Battery Capacity          1.000000     0.879453 -0.007829  0.046456 -
0.136900
Screen Size               0.879453     1.000000  0.108592  0.013568 -
0.285931
price                    -0.007829     0.108592  1.000000  0.404761
0.035998
storage                   0.046456     0.013568  0.404761  1.000000
0.118972
Camera                   -0.136900    -0.285931  0.035998  0.118972
1.000000
performance               0.999561     0.874175 -0.005514  0.051085 -
0.107608
value                     0.202748     0.067898 -0.714817 -0.354069 -
0.260434

                   performance     value
Mobile Weight         0.840624  0.054706
RAM                   0.156862 -0.528213
Front Camera         -0.139185 -0.324225
Back Camera          -0.077878 -0.193357
Battery Capacity      0.999561  0.202748
Screen Size           0.874175  0.067898
price                -0.005514 -0.714817
storage               0.051085 -0.354069
Camera               -0.107608 -0.260434
performance           1.000000  0.194370
value                 0.194370  1.000000
```

```
#as mobile weight is highly correlated with the screen and battery capacity so we would plot to see its relationship
plt.figure(figsize=(15,7))
sns.scatterplot(y=df["Screen Size"],x=df["Mobile Weight"],hue=df["Company Name"],size=df["Screen Size"])
plt.xlim(120,735)
plt.ylim(5,15)
plt.axhline(10,color="red")
plt.axvline(450,color="red")
plt.grid()
plt.show()
```




```
plt.figure(figsize=(15,7))
sns.scatterplot(y=df["Battery Capacity"],x=df["Mobile Weight"],hue=df["Company Name"],size=df["Battery Capacity"])
plt.axhline(6300,color="red")
plt.axvline(450,color="red")
plt.grid()
```



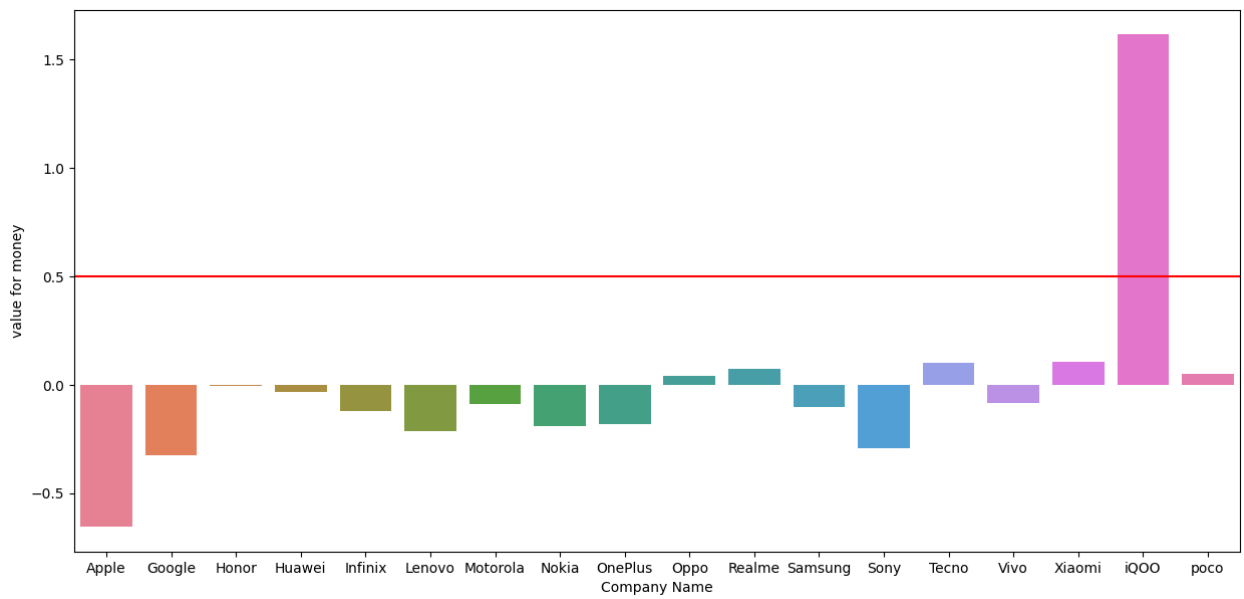
kpi

```
df

    Company Name  Mobile Weight  RAM  Front Camera  Back Camera  \
0          Apple          174.0    6            12           48
1          Apple          174.0    6            12           48
2          Apple          174.0    6            12           48
3          Apple          203.0    6            12           48
4          Apple          203.0    6            12           48
..           ...            ...  ...           ...          ...
925         poco          571.0    8             8            8
926         poco          571.0    8             8            8
927      Samsung          239.0   12            10           50
928      Samsung          239.0   12            10           50
929      Samsung          239.0   12            10           50

               Processor  Battery Capacity  Screen Size   price  \
0             A17 Bionic              3600          6.1   79999
1             A17 Bionic              3600          6.1   84999
2             A17 Bionic              3600          6.1   89999
3             A17 Bionic              4200          6.7   89999
4             A17 Bionic              4200          6.7   94999
..                   ...               ...          ...     ...
925  Snapdragon 7s Gen 2             10000         12.1   23999
926  Snapdragon 7s Gen 2             10000         12.1   25999
927   Snapdragon 8 Gen 3              4400          7.6  164999
928   Snapdragon 8 Gen 3              4400          7.6  176999
929   Snapdragon 8 Gen 3              4400          7.6  200999

     Launched Year  ... Camera  battery_per_inch  battery_weight_ratio
\
```

```
0             2024  ...     60        590.163934             20.689655

1             2024  ...     60        590.163934             20.689655

2             2024  ...     60        590.163934             20.689655

3             2024  ...     60        626.865672             20.689655

4             2024  ...     60        626.865672             20.689655

..             ...  ...    ...               ...                   ...

925           2024  ...     16        826.446281             17.513135

926           2024  ...     16        826.446281             17.513135

927           2024  ...     60        578.947368             18.410042

928           2024  ...     60        578.947368             18.410042

929           2024  ...     60        578.947368             18.410042


     performance     value   processor  camera_s  battery_s  display_a
\
0         1100.4  0.013755       apple -0.129446  -1.052660  -0.641802

1         1100.4  0.012946       apple -0.129446  -1.052660  -0.641802

2         1100.4  0.012227       apple -0.129446  -1.052660  -0.641802

3         1280.4  0.014227       apple -0.129446  -0.609796  -0.250378

4         1280.4  0.013478       apple -0.129446  -0.609796  -0.250378

..           ...       ...         ...       ...        ...        ...

925       3008.0  0.125339  Snapdragon -1.288000   3.671218   3.272437

926       3008.0  0.115697  Snapdragon -1.288000   3.671218   3.272437

927       1342.8  0.008138  Snapdragon -0.129446  -0.462175   0.336758

928       1342.8  0.007586  Snapdragon -0.129446  -0.462175   0.336758

929       1342.8  0.006681  Snapdragon -0.129446  -0.462175   0.336758


     value for money
0          -0.560117
1          -0.560117
```

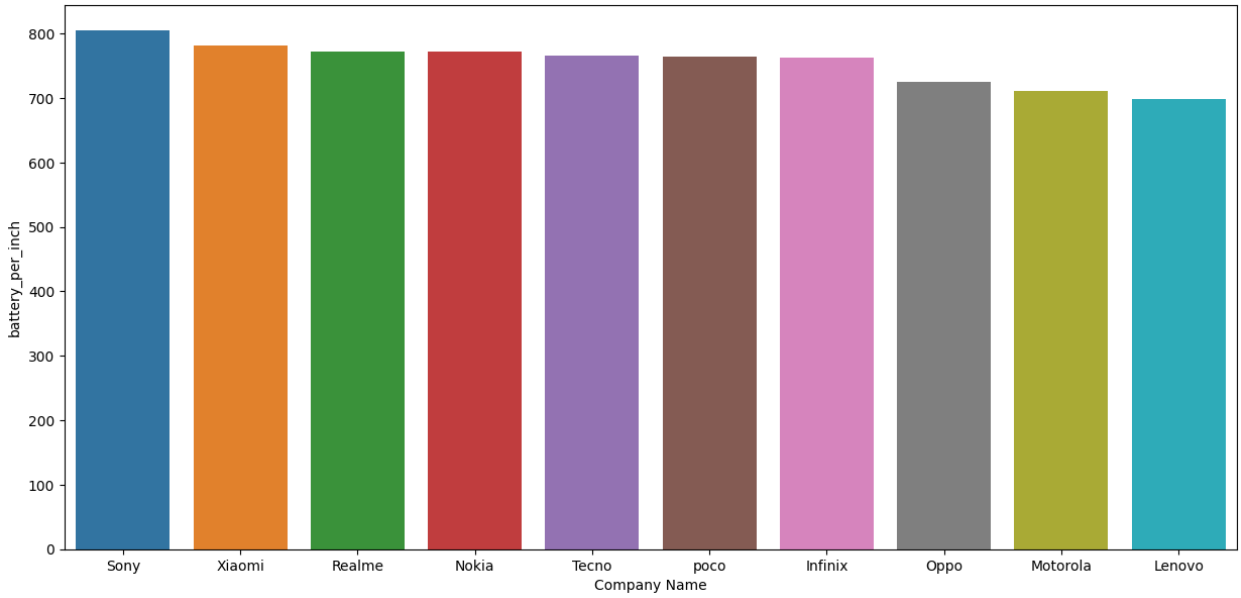
```
2           -0.560117
3           -0.309830
4           -0.309830
..                ...
925          1.567897
926          1.567897
927         -0.089403
928         -0.089403
929         -0.089403

[930 rows x 21 columns]
```

```
#which copamy offers the highest average value for money accros its models
kpi=df.groupby("Company Name")["value for money"].median().reset_index()
kpi1=pd.DataFrame(kpi1)
plt.figure(figsize=(15,7))
sns.barplot(x=kpi1["Company Name"],y=kpi1["value for money"],hue=kpi1["Company Name"])
plt.axhline(0.5,color="red")
plt.show()
```

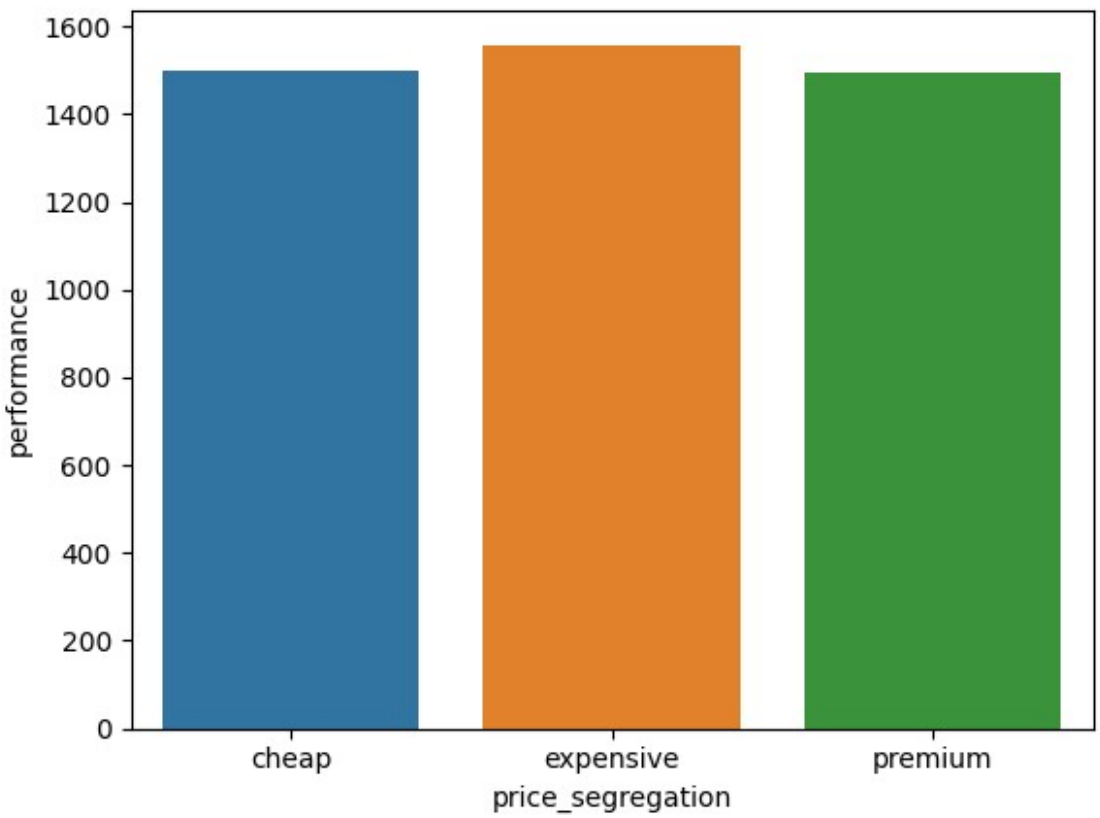



```
#top 10 phones by battery per inch
kpi=df.groupby("Company Name")["battery_per_inch"].mean().reset_index()
kpi2=pd.DataFrame(kpi2)
sort=kpi2.sort_values("battery_per_inch",ascending=False)[:10]
plt.figure(figsize=(15,7))
sns.barplot(x=sort["Company Name"],y=sort["battery_per_inch"],hue=sort["Company Name"])
```

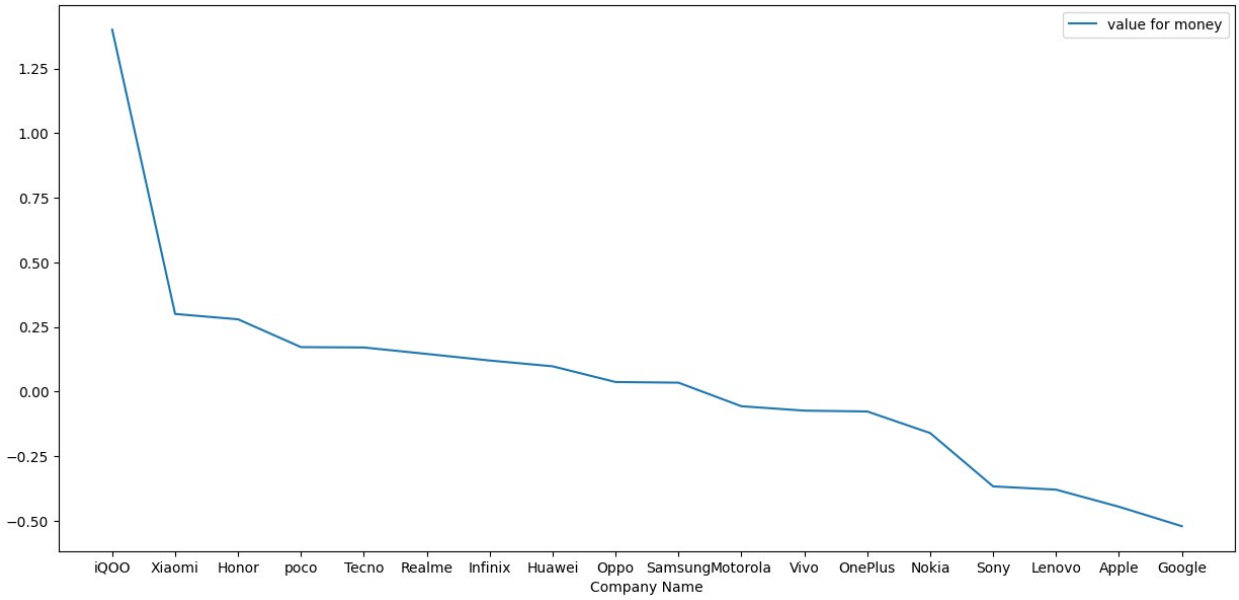
```
<Axes: xlabel='Company Name', ylabel='battery_per_inch'>
```



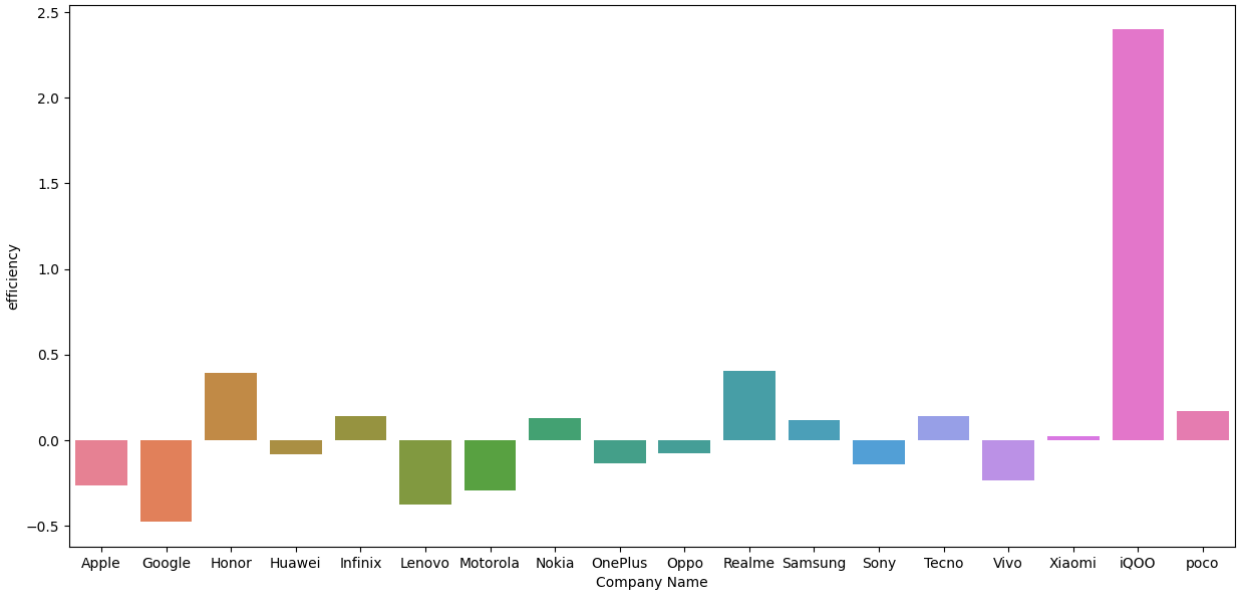

```
#as price increase do performance also increase
def price_seg(price):
    if price<20000:
        return "cheap"
    elif 20000<price<80000:
        return "expensive"
    else:
        return "premium"

df["price_segregation"]=df["price"].apply(price_seg)

kp3=df.groupby("price_segregation")
["performance"].mean().reset_index()
kp3=pd.DataFrame(kp3)
sns.barplot(x=kp3["price_segregation"],y=kp3["performance"],hue=kp3["p
rice_segregation"])
plt.show()
```




```
#which phone are the best overall considering camera battery and
displayt

kp4=df.groupby("Company Name")["value for money"].mean()
kp4=pd.DataFrame(kp4)
kp4_sorted=kp4.sort_values("value for money",ascending=False)

plt.figure(figsize=(15,7))
sns.lineplot(kp4_sorted,color="lightblue")

<Axes: xlabel='Company Name'>
```




```
#which phone deliver the best performence -battery-weight efficency

#first we need to standarize it
from sklearn.preprocessing import StandardScaler
scaler=StandardScaler()
df[["performence_s","Mobile
Weight_s"]]=scaler.fit_transform(df[["performance","Mobile Weight"]])

df["efficiency"]=0.5*df["performence_s"]+0.25*df["Mobile Weight_s"]
+0.25*df["battery_s"]

kp5=df.groupby("Company Name")["efficiency"].mean()
kp5=pd.DataFrame(kp5).reset_index()
plt.figure(figsize=(15,7))
sns.barplot(x=kp5["Company Name"],y=kp5["efficiency"],hue=kp5["Company
Name"])
plt.axhline()
plt.show()
```


2.5
2.0
1.5
1.0
0.5
0.0
−0.5
efficiency
Apple
Google
Honor
Huawei
Infinix
Lenovo
Motorola
Nokia
OnePlus
Oppo
Realme
Samsung
Sony
Tecno
Vivo
Xiaomi
iQOO
poco
Company Name